

PROJECT PPG
RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT (IRM)

CHOH JING YI	A23CS0296
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
GUI KAH SIN	A23CS0080
IMAN ABADI BIN MOHD NIZWAN	A23CS0084
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160
CHAU YING JIA	A23CS0213
EVELYN GOH YUAN QI	A23CS0222

UNIVERSITI TEKNOLOGI MALAYSIA

TABLE OF CONTENTS

TABLE OF CONTENTS	i
LIST OF TABLES	iv
LIST OF FIGURES	v
LIST OF ABBREVIATIONS	vi
LIST OF SYMBOLS	vii

TABLE OF CONTENTS

ABSTRACT	12
CHAPTER 1	13
INTRODUCTION	13
1.1 Introduction	13
1.2 Problem Background	13
1.3 Project Aim	14
1.4 Project Objectives	14
1.5 Project Scope	15
1.6 Project Importance	16
1.7 Report Organization	17
CHAPTER 2	18
RESEARCH METHODOLOGY	18
2.1 Introduction	18
2.2 Fundamental Theory & Concept	18
2.2.1 Supply Chain Management	18
2.2.2 Extract, Transform, Load (ETL/ELT)	19
2.2.3 Data Lake and Medallion Architecture	20
2.2.4 Data Warehouse and Star Schema	21
2.2.5 Inventory Management and Obsolescence	21
2.2.6 Data Visualization and Business Intelligence	22
2.3 Related Previous Researches/ Systems/ Compare between existing systems / Ensure strength and weakness between systems	22
2.3.1 Traditional Inventory Management Systems	22
2.3.2 Enterprise Resource Planning (ERP) Systems	22

2.3.3	Data-Driven Inventory Management Systems	23
2.3.4	Comparison of Existing Systems	23
2.4	Microsoft Azure	24
2.5	Microsoft Power BI	25
2.6	Summary	25
CHAPTER 3		27
METHODOLOGY		27
3.1	Introduction	27
3.2	Methodology Choice and Justification	27
3.3	Phases of the Chosen Methodology	28
3.3.1	Planning Phase	28
3.3.2	Analysis Phase	31
3.3.3	Design Phase	31
3.3.4	Implementation Phase	32
3.3.5	Testing Phase	33
3.4	Technology Used Description	33
3.5	Chapter Summary	34
CHAPTER 4		35
ANALYSIS AND DESIGN		35
4.1	Introduction	35
4.2	System Analysis	35
4.2.1	Case Study	35
4.2.2	System Requirements Gathering Techniques	36
4.3	System Requirements	37
4.3.1	Functional Requirements	37
4.3.2	Non-Functional Requirements	38
4.4	Current System Analysis	38

4.4.1	Current System and The Bottlenecks	39
4.4.2	Flow Chart for the Current Process	40
4.5	System Design	42
4.5.1	System Architecture	42
4.5.2	Component In System Architecture	44
4.5.3	System Design	46
4.5.3.1	Data Engineering Process	46
4.5.3.2	Data Source	46
4.5.3.3	Extraction, Transformation, and Loading (ETL)	47
4.5.3.4	System Process Design	50
4.5.4	Database Design	53
4.5.4.1	Entity Relationship Diagram (ERD)	53
4.5.4.2	Multidimensional Modelling	55
4.5.4.2	Dimensions Table	56
4.5.5	Interface Design	57
4.6	Summary	59
CHAPTER 5		61
IMPLEMENTATION AND TESTING		61
5.1	Introduction	61
5.2	System Development / Architecture	61
5.3	Coding of System's Main Functions	64
5.3.1	Silver Layer Data Transformation and Business Logic on Recoverable Assets (RA) Metrics and the Magna Carta (MC) Inventory Compliance	64
5.3.2	Silver Layer Data Transformation and Business Logic on Stockout Risks	70
5.3.3	Gold Layer Table	77
5.3.3.1	Database Initialisation	77
5.3.3.2	Dimensional Modeling Architecture	77

5.3.3.3 Dynamic Time Dimension Virtualization	79
5.3.3.4 Central Fact Table Ingestion	80
5.4 Essential Interfaces (The Dashboards & Tables)	81
5.4.1 Serverless SQL Views (Semantic Layer Design)	82
5.4.1.1 Semantic View Mapping and Logic Design Patterns	82
5.4.1.2 Semantic View Mapping and Logic Design Patterns	84
5.4.2 Power BI Dashboard Execution	85
5.4.2.1 Dashboard Page 1: Inventory Risk Overview	86
5.4.2.2 Dashboard Page 2: MC Obsolescence Analysis	88
5.4.2.3 Dashboard Page 3: Supply Chain Stockout Alert	90
5.5 Testing (Black box, White box, and User Acceptance)	92
5.5.1 Testing Summary & Sign-off	115
5.6 Summary	116
6.0 REFERENCES	117

LIST OF TABLES

Table 1: Report Organization Table	17
Table 2.1 : Comparison of Existing System	24
Table 4.3.1.1 : Functional Requirements	37
Table 4.3.2.1 : Non-Functional Requirements	38
Table 4.5.3.2.1 : Data Source Dictionary and Core Mapping	46
Table 4.5.4.1.1 Entities and Description of ERD	54
Table 4.5.4.1.2 Entity Relationships and Cardinality of ERD	55
Table 4.5.4.2.1 Dimensions Table in Star Scheme and Description	56
Table 4.5.4.2.2 Star Scheme Relationship and Cardinality	57

LIST OF FIGURES

Figure 2.2.2.1: Example of ELT and ETL	19
Figure 2.2.3.1 : Medallion Architecture	20
Figure 2.2.4.1 : Example of Star Schema	21
Figure 3.3.1.1 : Gantt Chart 1	29
Figure 3.3.1.2 : Gantt Chart 2	30
Figure 3.3.1.3 : Gantt Chart 3	30
Figure 4.4.2.1 Current Manual Inventory Risk Management Flow Chart	41
Figure 4.5.1.1 : End-to-end System Architecture	42
Figure 4.5.1.2 : System Medallion Layer Data Flow Architecture	43
Figure 4.5.3.4.1 : Use Case Diagram	50
Figure 4.5.3.4.2 : Activity Diagram	51
Figure 4.5.3.4.3 : Sequence Diagram	52
Figure 4.5.4.1.1 Normalized ERD of PPG Inventory Management System	53
Figure 4.5.4.2.1 Star Scheme for Inventory Dashboard System	56
Figure 4.5.5.1 Interface Design for Asset Recovery and Active Inventory Status Dashboard	58
Figure 4.5.5.2 Interface Design for Magna Carta (MC) Financial Provisions Dashboard	59
Figure 4.5.5.3 Interface Design for Customer Order Delivery Risks Dashboard	59
Figure 5.2.1 Azure Data Factory pipeline for extracting raw CSV files into the Bronze Layer	62
Figure 5.2.2 Azure Data Factory Mapping Data Flow for Data Quality Validation and Duplicate Detection	63
Figure 5.3.1.1 : Silver Layer Source Ingestion Configuration	65
Figure 5.3.1.2: Recoverable Assets (Q2) Pipeline Logic	66
Figure 5.3.1.3: Console Sample View of Q2	66
Figure 5.3.1.4: Compilation of Magna Carta Dead Stock Flags	67

Figure 5.3.1.5: Console Sample View of Q3	67
Figure 5.3.1.6: Valuation and Processing of Expired Lots and Financial Write-Down Provisions	68
Figure 5.3.1.7: Console Sample View of Q4	69
Figure 5.3.1.8: Console Sample View of Q5	69
Figure 5.3.1.9: Save into Silver File	70
Figure 5.3.1.10: Snapshot of Silver File in Azure Storage	70
Figure 5.3.2.2 Reorder Level Detection Code Snippet	71
Figure 5.3.2.3 Reorder Level Detection Output	72
Figure 5.3.2.4 Stockout Risk Calculation Code Snippet	73
Figure 5.3.2.5 Stockout Risk Calculation Output	73
Figure 5.3.2.6 Sale Impact Tracing Code Snippet	74
Figure 5.3.2.7 List of Impacted Orders Output	75
Figure 5.3.2.8 List of Affected Customers Output	75
Figure 5.3.2.9 Writing Output to Silver Code Snippet	76
Figure 5.3.3.1 : Database Initialisation Source Code	77
Figure 5.3.3.2: Dimensional Modeling Architecture (dim_material, dim_supplier, and dim_customer)	78
Figure 5.3.3.3 : dim_date Source Code	80
Figure 5.3.3.4 : Database Initialisation Source Code	81
Figure 5.4.2.1 Inventory Risk Overview Dashboard	86
Figure 5.4.2.2 MC Obsolescence Dashboard	88
Figure 5.4.2.3 Supply Chain Stockout Alert Dashboard	90

LIST OF ABBREVIATIONS

RA	-	Recoverable Asset
MC	-	Magna Carta
CETAS	-	Create External Table As Select
ETL	-	Extract, Transform, Load
ERP	-	Enterprise Resource Planning
ACID	-	Atomicity, Consistency, Isolation, & Durability
ADF	-	Azure Data Factory

LIST OF SYMBOL

ADC_m	-	Average Daily Consumption of material m
$C_{m,t}$	-	Production consumption volume of material m at month t
$C_m(9)$	-	Cumulative production consumption of material m over a 9-month window
$C_m(12)$	-	Cumulative production consumption of material m over a 12-month window
$D_{current}$	-	System runtime execution date
D_{expiry}	-	Physical lot or batch expiration date
I_m		Current physical inventory stock quantity on hand for material m

ABSTRACT

Nowadays, it is crucial for manufacturing businesses to manage their inventory effectively to avoid financial losses and ensure the smooth functioning of the supply chain. PPG faces challenges including inventory obsolescence, recoverable assets, Magna Carta (MC) dead stock, expired materials and stockout risks that could impact customer order fulfillment. Moreover, there is no integrated data pipeline to enable data from the operations to be converted into meaningful insights for decision making.

The goal of this project is to create an end-to-end data engineering pipeline and interactive data analytics dashboard for Recoverable Assets (RA) and Inventory Risk Management (IRM). The project leverages 6 operational datasets which are materials, inventory snapshots, production orders, purchase orders, sales orders and suppliers. A three-layer Medallion Architecture including Bronze, Silver, and Gold layers is implemented using Microsoft Azure services, such as Azure Data Factory, Azure Data Lake Storage Gen2, and Azure Synapse Analytics. Data ingestion, validation, cleaning, transformation, and business rule implementation are performed to identify recoverable assets, Magna Carta dead stock, expired materials, financial provisions, and stockout risks. The data is then transformed into a star schema structure and visualized through Microsoft Power BI dashboard.

The developed solution automatically performs the Inventory Risk Analysis and offers analytics ready datasets for reporting and decision making. The interactive dashboards allow stakeholders to track inventory health, analyse financial risk, uncover materials at-risk and understand the potential effect of stock shortages on customer sales orders. Overall, the proposed system enhances the visibility of data, decreases manual effort, and facilitates data-driven inventory risk management in the organization.

CHAPTER 1

INTRODUCTION

1.1 Introduction

This project focuses on designing and implementing an end-to-end data engineering pipeline to support supply chain management and improve business decision making. The project collaborates with PPG, a world's second largest coating company by revenue. It produces paints, coatings, optical products and specialty materials for industrial, aerospace, and architectural markets that serve both industrial and consumer markets. The main objective of this project is to address operational challenges related to Recoverable Assets (RA) and Inventory Risk Management (IRM) to improve financial performance and operational efficiency.

This project uses a medallion data architecture which contains bronze, silver, and gold layers to improve data quality and data consistency. The architecture enables raw data from multiple sources to be collected, processed and transformed into useful business insights. This project uses Microsoft Azure for data ingestion, processing, and transformation while Microsoft Power BI is used to create interactive dashboards for visualization. This helps managers have clear insights into inventory risks, recoverable asset performance, and material availability. These insights help managers do informed decision-making and help ensure that the company has enough materials to fulfill customer sales orders.

1.2 Problem Background

For the coatings and paints sector, inventory control plays a crucial role as many raw materials like pigments, resins, and solvents have an expiration date and need to be consumed within a designated time frame. If these materials are not monitored carefully, these materials might expire before they are used leading to waste and losses. PPG is a global manufacturer with a

significant amount of inventory in its operations and inventory control is a very challenging but crucial component for its business.

PPG currently has a number of challenges to ensure an efficient and responsive supply chain. These can impact inventory usage, cost of operations and overall business performance. The following key problems have been identified:

1. **Financial Risk due to Inventory Obsolescence:** A large size of raw materials is categorized as Recoverable Assets (RA) when there are more than 12 months of consumption. Moreover, the items that fall into the category of the Magna Carta (MC) logic called the Dead Stock when there is no consumption for 9 months and the Expired lots must have the financial write-down of 100%. This would be inefficient to quantify these total amounts of provision without an automated system.
2. **Lack of Integrated Data Pipeline:** Currently, PPG's operational data, such as materials, inventory snapshots, purchase orders, and sales orders, are present in individual synthetic data. There is a lack of an end-to-end data engineering pipeline to ingest, clean and transform this raw data into a clean, analytics ready format.
3. **Fulfillment and Stockout Risks:** There is no process to streamline the identification of materials at risk of stockout relative to their lead times. This prevents the organization from tracing exactly which open customer sales orders will be affected when a raw material shortage occurs, resulting in potential delivery failures and customer dissatisfaction.

1.3 Project Aim

The aim of this project is to develop a data engineering pipeline and an interactive analytics dashboard using Microsoft Azure and Power BI. The tool helps identify recoverable assets and address financial risks associated with expired inventory.

1.4 Project Objectives

Implementing this data engineering pipeline and analytical framework provides a complete, end-to-end solution for managing inventory risks. By mapping clear metrics directly to the project's goals, the system automates how the company spots recyclable assets and catches

obsolescence losses, turning raw data into clear, actionable business intelligence. The project was completed by hitting four key objectives:

- i. **Analyze the Data:** Examined the supply chain data across the six source CSV files to pinpoint the exact attributes needed to track Recoverable Assets (RA), Magna Carta (MC) classifications, and stockout risks.
- ii. **Design the Architecture:** Planned a robust, three-layer data architecture. This included setting up clear ingestion methods, mapping out the transformation steps for the RA/MC business logic, and creating a star schema data model built specifically for risk analytics.
- iii. **Build the Pipeline:** Developed the actual end-to-end pipeline inside Microsoft Azure to automate raw data ingestion, handle thorough cleaning and transformations, and output analytics-ready datasets.
- iv. **Deliver and Validate:** Built an interactive Power BI dashboard that directly answers the seven core business questions (Q1–Q7) while verifying that the entire system is accurate and performs well.

1.5 Project Scope

This project was designed with the goal of creating a data engineering pipeline that transforms synthetic operational data into business intelligence that can be used to make business decisions for inventory risk management. The key elements of this project is as follows:

(a) Build a Three-Layer Data Pipeline

This project involves setting up a structured system using Microsoft Azure to process the six synthetic data files. The process will start with the Bronze Layer that is responsible with the initial data upload and basic cleanup like removing duplicates. The process is then proceeded to the next process, which is Silver Layer to perform complex calculations, such as identifying materials that have not been used for a long time. Finally, the Gold Layer will organize and put all the information into a "star schema" format within a data warehouse, making the information ready for final reporting.

(b) Apply Inventory Risk & Financial Business Logic

The aim of this project is to automatically classify the materials based on the specific industry risk categories that includes identifying Magna Carta (MC) Dead Stock (materials with no consumption for over 9 months) and MC Expired materials (those past their lot expiry date). The project also quantifies the total financial provision required for these materials and identifies stockout risks by comparing current stock availability and lead times.

(c) Analytics Dashboard & Impact Reporting

The project ends with the development of the dashboard that visualizes the pipeline's results. This dashboard is designed to answer the key operational questions by linking the processed data to a visual tool as the project will show how the automated tracking helps stakeholders monitor inventory health and customer impact in real-time.

1.6 Project Importance

The importance of this project relies on the ability of the pipeline to connect the gap between raw material data and business logic in producing an analytical dashboard model. This project will deliver a clean “analytical-ready” data model to stakeholders and steering committee to meet the goal with the following properties:

(a) Mitigating Financial Risk

To implement the automation of the detection of Magna Carta (Dead Stock and Expired) materials that enables the company to accurately measure financial provisions and write downs which provide transparency in accounting.

(b) Optimisation of Asset Recovery

Helps the industry (PPG) to reduce waste and enhance cash flow by identifying materials that are more than 12 months consumption (RA).

(c) Protects Customer Relationships

The system will give an early warning indication on customers' expectations so that it can manage the risks in an open sales order and prevent loss from the sales.

1.7 Report Organization

This report is organized into five chapters to provide a structured presentation of the project development process, from business problem to system implementation and evaluation.

Table 1: Report Organization Table

Overview of Report Organization		
Chapter 1	Introduction	This chapter provides an overview of the project, including the project's background, aims, objectives, scope, significance and overall structure of the report.
Chapter 2	Literature Review	This chapter reviews the relevant literature and theoretical concepts related to the project. It provides the theoretical concepts, technologies used as well as support the proposed solution.
Chapter 3	Research Methodology	This chapter describes the methodology adopted in conducting the project. It explains the development approach, project phases, techniques and technologies used throughout the implementation process.
Chapter 4	Analysis and Design	This chapter focuses on system analysis and design of the proposed system. It discusses the functional and non-functional requirements, current system analysis, system architecture, database design and interface design.
Chapter 5	Implementation and Testing	This chapter describes the system development process, coding implementation, system interfaces, and testing methods such as black-box, white-box, and user testing.

CHAPTER 2

RESEARCH METHODOLOGY

2.1 Introduction

This chapter aims to explore deeper in Cloud Computing and Supply Chain Management and also make a deeper understanding of the foundation for this project. This chapter will provide a comprehensive understanding on the principles and technologies needed in order to establish a solid foundation for the data pipeline, such as the Medallion Architecture, ETL (Extract, Transform, Load) processes and modern Data Lakehouse environments.

Moreover, several past existing inventory management strategies will be analyzed in this chapter, with an emphasis on inventory obsolescence and the financial consequences of managing Recoverable Asset (RA) and Magna Carta (MC) materials. To support the proposed solution, a comparison of current systems from simple tracking to more sophisticated Enterprise Resource Planning (ERP) systems is done and demonstrates the need for a scalable, data-driven solution. Last, the recommended technologies to be used will be explained, such as Microsoft Azure services and Power BI in order to show how it can be brought together to transform raw synthetic data into business intelligence.

2.2 Fundamental Theory & Concept

The design of this pipeline and analytics solution comes down to three main things: data engineering, data management, and solid inventory control. Focusing on these core basics ensures that the final architecture is both technically efficient and genuinely useful for daily business operations.

2.2.1 Supply Chain Management

Supply Chain Management (SCM) is all about coordinating the flow of materials, information, and money from procurement right through to final delivery (Chopra & Meindl,

2016). The ultimate goal is to keep things running efficiently while keeping operational costs as low as possible. In this project, the focus is squarely on inventory control, supplier alignment, and order fulfillment which the exact areas where friction, like holding too much stock or running out of items, turns into direct financial losses.

To stay competitive, companies rely on automated data pipelines to bring together data from inventory systems, purchase orders, and sales transactions. Automatically moving and preparing this data for analytics removes blind spots, making it much easier to monitor risks and make quick, informed decisions.

2.2.2 Extract, Transform, Load (ETL/ELT)

Extract, Transform, Load (ETL) is a key process in data warehousing that integrates data from multiple sources into a centralized system for analysis. It involves extracting data, transforming it into a structured and consistent format and loading it into a data warehouse. According to Panos Vassiliadis (2009), ETL processes also include data cleaning and transformation to resolve inconsistencies and ensure data quality before loading into the warehouse. Additionally, ETL supports the regular updating of data to provide accurate and up-to-date information.

An alternative approach is Extract, Load, Transform (ELT), where raw data is first loaded into the storage system before transformation is applied. This approach is commonly used in modern data platforms where processing power is shifted to the data storage layer. Figure 2.2.2.1 shows the illustration of both approaches ELT and ETL.

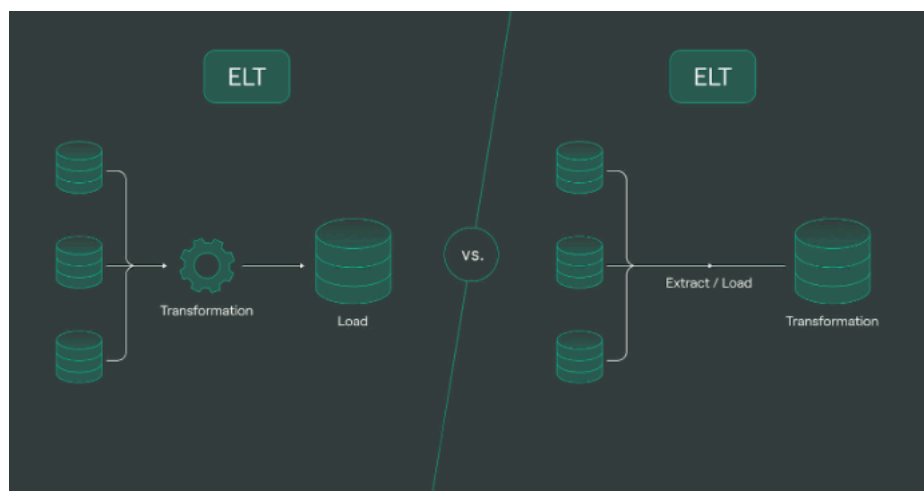


Figure 2.2.2.1: Example of ELT and ETL

In this project, the ETL/ELT process is used to integrate multiple supply chain datasets, perform data cleaning and transformation and implement business rules such as Recoverable Assets (RA) and Magna Carta (MC) classifications.

2.2.3 Data Lake and Medallion Architecture

A data lake is used to store raw data in its original format, allowing flexibility for future processing. It is designed to handle large volumes of structured, semi-structured and unstructured data without requiring strict schema definitions at the point of ingestion. This makes it particularly suitable for modern data-driven systems where data comes from multiple heterogeneous sources and may need to be reused for different analytical purposes in the future.

To improve data organization and processing efficiency, this project adopts the Medallion Architecture, which consists of three layers, the Bronze layer represents raw ingested data, the Silver layer contains cleaned and transformed data, and the Gold layer provides aggregated and analytics-ready datasets (Armbust et al., 2021). Each layer plays a specific role in cleaning and structuring the data as it moves through the pipeline. The Bronze layer serves as the raw landing zone, keeping the files in their original format so we don't lose any data during ingestion. From there, the Silver layer takes over to clean, standardize, and integrate the data, making it consistent and actually usable. Finally, the Gold layer handles the business-level aggregations, optimizing the datasets so they are easy to query and ready for decision-making.

Figure 2.2.3.1 breaks down the project's Medallion Architecture, tracking how the data moves from the raw Bronze layer, through the refined Silver layer, and into the final Gold layer for analytics and reporting.



Figure 2.2.3.1 : Medallion Architecture

2.2.4 Data Warehouse and Star Schema

At its core, a data warehouse is a centralized system built specifically for running analytical queries and generating reports. Unlike a standard operational database that handles day-to-day transactions, a data warehouse pulls data from different sources and structures it specifically for decision-making. This setup makes it much easier to process massive amounts of historical data, which is exactly what is needed for trend analysis, forecasting, and business intelligence.

For the layout, a star schema is used to connect a central fact table to multiple dimension tables, making analytical queries a lot more efficient (Kimball & Ross, 2013). This setup is widely used because it keeps things simple and speeds up performance by cutting down on complex table joins. The fact table holds all the numerical data and measurable metrics, while the dimension tables store the descriptive details like time, location, or product categories that give context to those numbers. Figure 2.2.4.1 shows the example structure of the star schema used in this project.

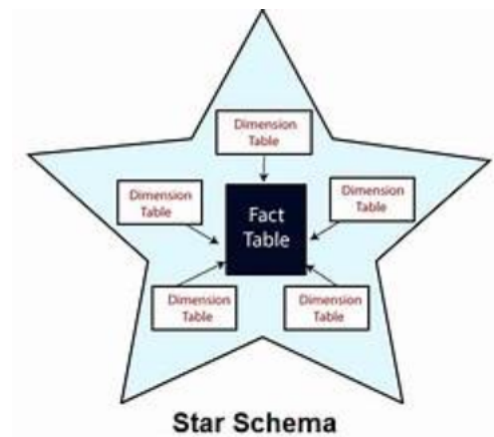


Figure 2.2.4.1 : Example of Star Schema

2.2.5 Inventory Management and Obsolescence

In inventory management, obsolescence happens when materials lose their value because market demand shifts or the products simply expire. Under this framework, Recoverable Assets (RA) are essentially surplus stock where items on hand that exceed what the company actually expects to consume. On the other hand, Dead Stock and expired materials represent dead capital, which means they are just sitting there and require financial write-downs due to inactivity or passed shelf-life limits (Silver, Pyke & Thomas, 2016). Keeping a close eye on these

classifications is essential for cutting financial losses and keeping the overall supply chain running tightly.

2.2.6 Data Visualization and Business Intelligence

Data visualization tools, especially Microsoft Power BI, turn complex data structures into easy-to-use, interactive dashboards. This visual approach lets decision-makers spot trends, isolate operational risks, and track performance metrics at a glance, making strategic planning a lot faster and more straightforward (Few, 2012).

2.3 Related Previous Researches/ Systems/ Compare between existing systems / Ensure strength and weakness between systems

The number of systems and the development of research studies has emerged to address the challenges of inventory management and address the risk of operations. These systems allow stock levels to be closely monitored, limit overstocking and prevent stock shortages. Three systems are discussed in this section and their pros and cons with respect to the project are compared.

2.3.1 Traditional Inventory Management Systems

Organizations usually use traditional inventory management systems to keep track of stock levels and manage the materials. Usually, they are simple manual or semi-automated in a spreadsheet or simple database. The key benefit of traditional systems is they are simple and inexpensive to implement. However, they are highly dependent on human input, which increases the likelihood of human errors and data inconsistency. Besides, these systems do not process data in real-time and are not capable of processing large datasets containing multiple departments.

2.3.2 Enterprise Resource Planning (ERP) Systems

In large organizations, Enterprise Resource Planning (ERP) like SAP is extensively implemented to connect different business processes. For instance, inventory management, procurement and sales sections. ERP systems ensure real-time data updates, data centralization and enhanced transparency between departments. These features enable organizations to make better decisions and improve operational efficiency. At the same time, ERP systems are costly to

deploy and operate as they demand a higher level of expertise and do not support the implementation of complex analytics or custom business rules like inventory risk prediction.

2.3.3 Data-Driven Inventory Management Systems

Over the past few years, numerous organizations have turned to using data to enhance inventory management. These systems use data analytics and automated pipelines to process large volumes of data rather than manually process. By using data transformation, aggregation and visualization techniques, businesses can gain insights that are meaningful to support better decision making. The advantage of these systems is that they are able to track large amounts of data efficiently and gain insight into inventory trends. However, they also require technical expertise and proper system design to ensure data accuracy and reliability.

2.3.4 Comparison of Existing Systems

The existing systems have been compared and it is found that each has its own advantages and drawbacks. Traditional inventory systems are easy to implement and affordable, which makes them suitable for small businesses. However, manual processes make it difficult for them to ensure real-time inventory data and increase the possibility of human error. As the volume of inventory data grows, these systems become increasingly difficult to maintain and scale.

Many of these challenges can be addressed by the ERP system, in addition to inventory management, it can also be linked to other business related areas like procurement and sales. They provided centralized data storage and real-time data sharing, which improved data visibility, consistency and decision making of the operations. However, ERP systems are expensive to implement and maintain because they are complex to use and require special knowledge. In addition, they are often developed for a wide range of business functions and do not necessarily offer specific analytics for inventory risk management.

Besides, data-driven inventory systems are a more sophisticated method that involves automated data pipelines and analytics tools to efficiently analyze large amounts of data. These systems are used for advanced analysis, such as inventory risk detection, trend analysis and

business intelligence reporting. The implementation requires technical skills and careful design of the system to maintain the quality and reliability of data.

Table 2.1 : Comparison of Existing System

System	Description	Strengths	Weaknesses
Traditional Inventory System	Manual or basic system used to track stock levels	Simple, low cost, easy to implement	High risk of human error, no real-time updates, limited scalability
ERP System	An integrated system managing multiple business processes including inventory	Real-time data, centralized system, improved visibility	High cost, complex implementation, limited flexibility for advanced analytics
Data-Driven Inventory System	Automated system using data pipelines and analytics tools	Scalable, efficient processing, supports advanced analysis	Requires technical expertise, higher system complexity

From the above comparison, it shows that traditional systems are not automated and lack accuracy, whereas ERP is powerful but expensive and is not particularly designed for advanced inventory risk analysis. To overcome these limitations, the project aims to develop a data engineering pipeline approach that integrates automation, scalability and advanced data processing techniques. The system is used to monitor Recoverable Assets (RC), Magna Carta (MC) Dead Stock, expired materials and stockout risks, making it more efficient and data-driven to manage stocks.

2.4 Microsoft Azure

A study by Sharma and Panda (2023) suggested using Azure for supply and stock risk management by offering integrated infrastructure, software and platform level services that support multiple supply chain and warehouse operations where the platform improves both operational efficiency and financial performance. The study supports that third-party logistics providers are using public and hybrid clouds more frequently especially for automation and cybersecurity needs. However, potential challenges like vendor dependency and security exposure are a few concerns that should be considered. The study supports the use of Azure services in the architecture for the current project which uses MS Power BI, Azure Synapse and Azure Data Factory to create an end-to-end Medallion pipeline for the PPG case study.

2.5 Microsoft Power BI

Power BI is a business intelligence tool that enables organizations to visualize data and share insight by using interactive dashboards. It can connect with other Microsoft services, pulling data to change it from raw data into powerful, actionable insights to better understand hidden patterns and trends. (Daruvuri et al., 2024)

2.6 Summary

The literature review highlights the importance of building an end-to-end data pipeline for obsolescence detection that is scalable, flexible and robust. The fundamental theory and concepts provide the foundational knowledge required, such as ETL, a data integration process that is crucial before data analysis can be done. Furthermore, concepts such as data warehouses and data lakes help in understanding their differences and how the use case differs depending on the requirement of the data pipeline.

Previous studies of inventory management systems show different approaches used to reduce operational risks. Traditional management systems only have basic functionalities, limiting advanced analytics. ERP systems, on the other hand, provide real-time data and centralization but are not financially feasible most of the time. A data-driven system was chosen because it offers automation, scalability, flexibility and advanced preprocessing which is suitable for inventory and stock risk management as it requires advanced analytics. The use of

cloud technologies such as Microsoft Azure shows how these services can help support building an end-to-end pipeline.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter describes methodology used to build the data engineering pipeline for PPG. It is crucial in planning the project to ensure successful implementation. The chosen approach is the Data Engineering Lifecycle supported by the System Development Life Cycle (SDLC). The project is broken down into five phases. Planning to set the goals and schedule; Analysis to understand the project requirements and business rules; Design to plan the end-to-end data pipeline architecture; Implementation to build the actual pipeline in Microsoft Azure and create the Power BI dashboard; and testing to ensure the data is accurate and error-free. By following this structured plan, we ensure the final system is built successfully and meets all project objectives.

3.2 Methodology Choice and Justification

Data Engineering Lifecycle is the main methodology used in this project which is supported by the System Development Life Cycle (SDLC). The choice of this methodology is appropriate for this project as it sets out to develop a full end-to-end solution for the data pipeline for inventory risk management and analytics.

The Data Engineering Lifecycle is appropriate for this project as it goes through multiple processes of data processing, including data ingestion, transformation, storage, and visualization (Reis & Housley, 2022). This is further enhanced by implementing the three layer architecture including Bronze, Silver and Gold layers which adheres to Medallion Architecture principles, whereby data gradually moves from its raw state to analytics-ready datasets. The Data Engineering Lifecycle is a structured and systematic approach to managing data that makes it more suitable than general software development methodologies for achieving the goals of this project.

In addition, the project will require handling several types of data and applying complex business logic such as Recoverable Assets (RA), Magna Carta (MC) Dead Stock and Expired materials. That is why a data-centric approach that ensures the quality, consistency and scalability of data across the data pipeline is necessary. Furthermore, because of the technologies that were leveraged, including Microsoft Azure technologies such as Azure Data Factory, Azure Data Lake storage, Azure Synapse Analytics and Power BI, this selection also fits in with the modern data engineering processes.

The SDLC is included to provide a structured way to develop through stages such as Planning, Analysis, Design, Implementation and Testing. This ensures the project development is systematic, well structured and aligned with the project goals. Therefore, the selected methodology is a sensible approach as it can help with the data processing requirements and the design of the scalable and stable data engineering pipeline.

3.3 Phases of the Chosen Methodology

This section details the operational steps taken across each major project phase which are Planning, Analysis, Design, Implementation, and Testing.

3.3.1 Planning Phase

The planning phase is all about setting the overall direction, feasibility, and technical boundaries for the project. The main goal here was to build an end-to-end data engineering pipeline using Microsoft Azure and Power BI to support inventory risk management. This initial stage kept the development work aligned with the actual business problems we needed to solve, while staying within our time and resource limits.

Looking at the operational challenges from Chapter 1 like inventory obsolescence, financial liabilities, and the lack of a joined-up data infrastructure, the scope was tailored around a few key deliverables. These included a three-layer Medallion architecture (Bronze, Silver, Gold), built-in transformation logic for Recoverable Assets (RA), Magna Carta (MC), Dead Stock, and Expired Materials, and a final interactive Power BI dashboard.

To make sure everything was delivered on time, the project lifecycle was mapped out using a Gantt chart. The resulting schedule and phase durations are shown below in Figures 3.3.1.1, 3.3.1.2, and 3.3.1.3.

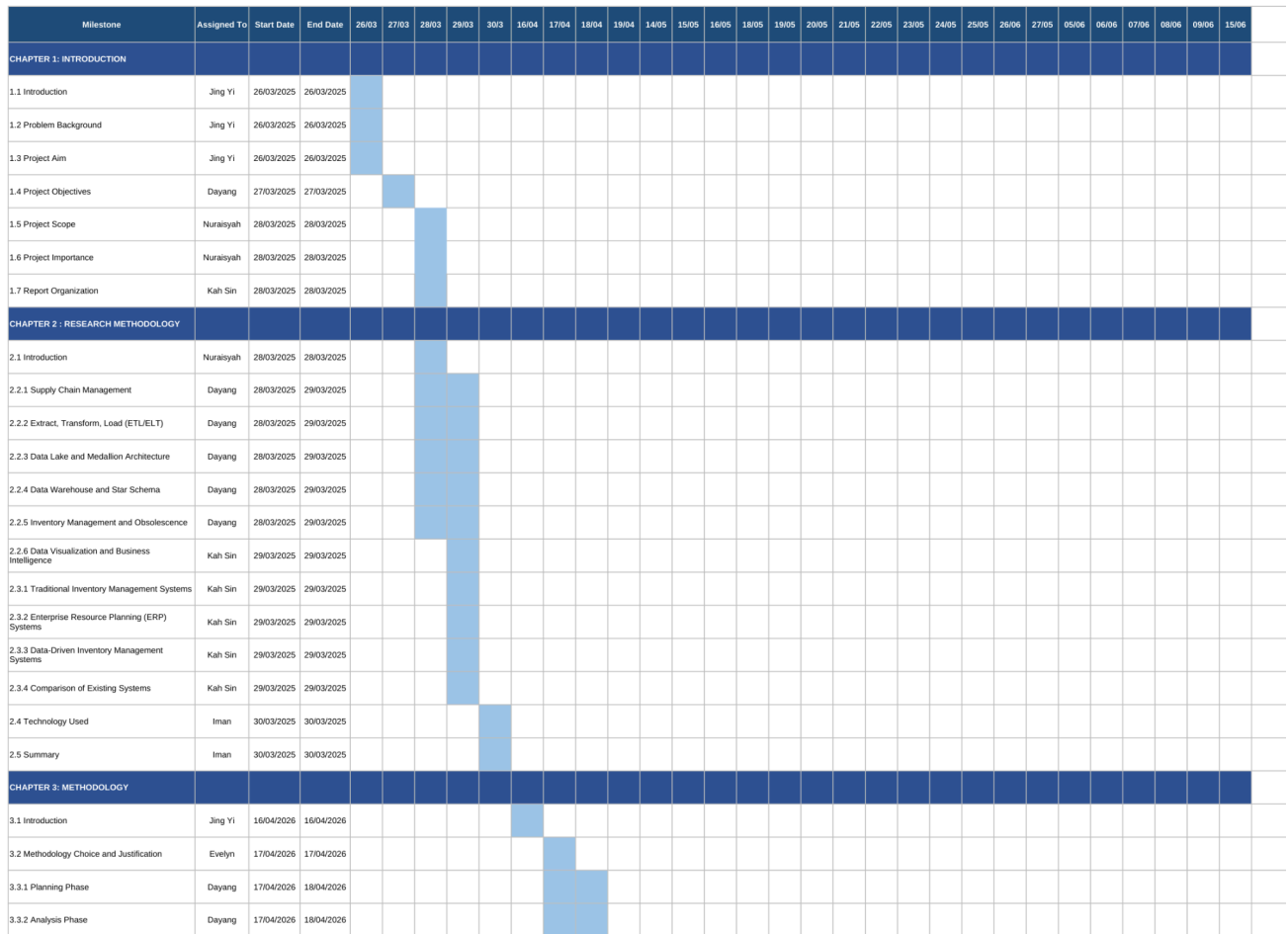


Figure 3.3.1.1 : Gantt Chart 1

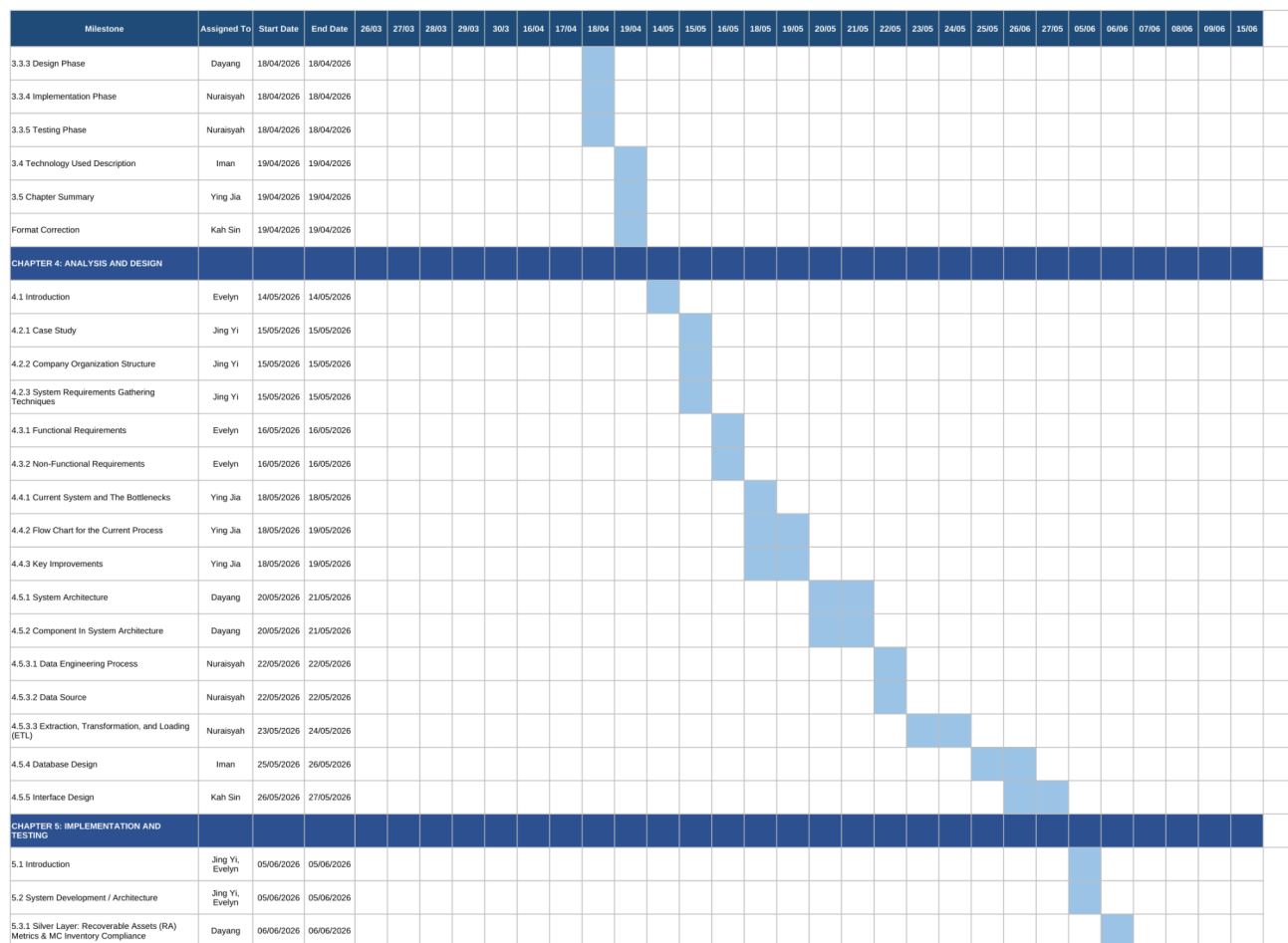


Figure 3.3.1.2 : Gantt Chart 2

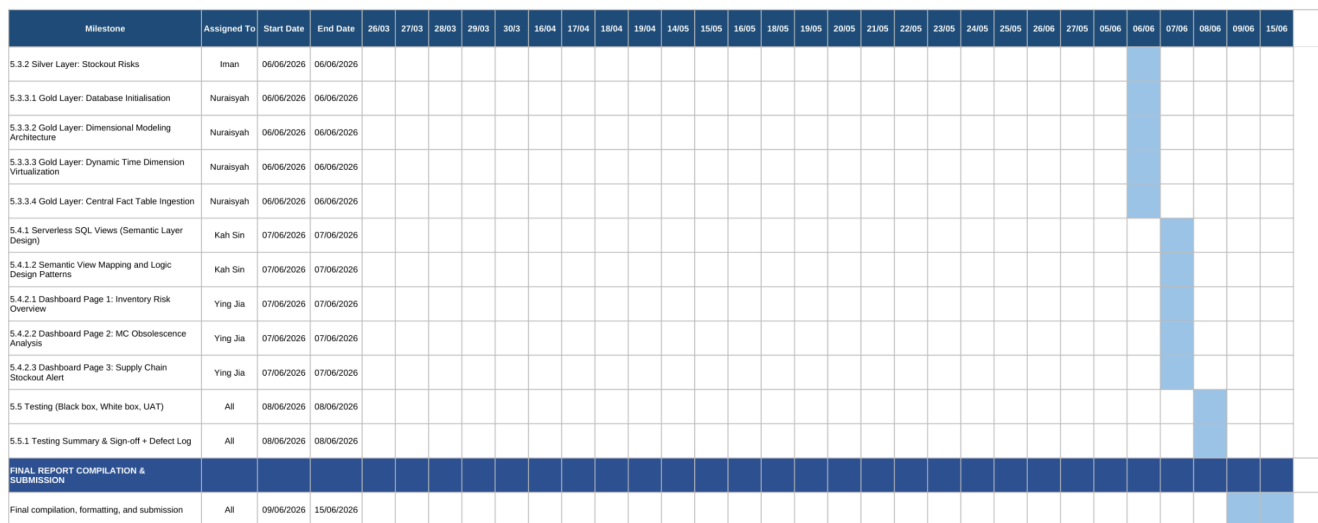


Figure 3.3.1.3 : Gantt Chart 3

3.3.2 Analysis Phase

The analysis phase is where the system's functional and non-functional requirements are laid out. On the functional side, the architecture needs to take in various data streams, clean them up, apply inventory risk rules, and deliver datasets that are completely ready for analysis. The platform also has to support interactive dashboards in Power BI so users can easily dig into the data and answer core business questions.

From a non-functional standpoint, the focus is on scalability, reliability, and precision. To handle multi-layered data pipelines efficiently, the infrastructure needs to deal with growing data volumes without slowing down. Built-in data integrity checks are also included to ensure the final analytical reports are completely accurate and trustworthy.

Another big part of this phase is locking down the exact business logic. The transformation rules state that any inventory sitting above a 12-month consumption threshold gets flagged as a Recoverable Asset. Meanwhile, Dead Stock and expired items are isolated based on shelf-life limits and a total lack of movement. These specific rules form the core of the data transformation layer, ensuring the system accurately catches real-world inventory risks.

3.3.3 Design Phase

System development is built on a three-tier Medallion Architecture running on Microsoft Azure cloud infrastructure. Within the data layer, the Bronze stage takes in the raw source files without changing their structure, which gives us an unedited audit trail of the incoming data. The Silver stage is where data quality controls are introduced. Here, cleaning routines purge duplicates, fix formatting issues, and handle missing values. Crucially, the transformation pipelines at this layer apply the mathematical rules needed to isolate Recoverable Assets, Magna Carta metrics, Dead Stock, and Expired items.

The final optimizations happen in the Gold Layer, where the data is structured into a star schema model. By connecting a central transactional fact table (inventory status) to descriptive dimension tables (materials, calendar, and suppliers), the system simplifies complex joins and speeds up performance. The operational backbone of this pipeline relies on a few key cloud services:

- Azure Data Factory: Handles the end-to-end workflow orchestration and automates the ingestion triggers.
- Azure Data Lake Storage: Provides scalable, organized storage folders for the Bronze, Silver, and Gold environments.
- Azure Synapse Analytics: Supplies the heavy computational processing power needed for large-scale queries and transformations.
- Power BI: Serves as the presentation layer, turning those final Gold tables into interactive business intelligence dashboards.

3.3.4 Implementation Phase

In implementation phase, an actual construction of data pipeline and development of analytical dashboard executed through the key activities below:

- **Data Ingestion (Bronze Layer):**

The pipelines are developed in Azure Data Factory to ingest the six CSV files into the Bronze Layer of Azure Data Lake Storage, where in this stage, it will focus on maintaining the data fidelity. By linking the services and the dataset, the automation will configure the movement of raw data from the source to the landing zone.

- **Data Transformation (Silver Layer):**

By utilizing Azure Synapse Analytics, the raw data is cleaned and refined, which the process includes removing the duplicate and handling null data, coding the Magna Carta logic to identify the expired materials, and calculate the Recoverable Assets (based on 12 months consumption threshold).

- **Data Modeling (Gold Layer):**

The data that has been transformed is aggregated in a Star Schema. Fact tables and the dimensions table are written by ensuring ACID transactions (Atomicity, Consistency, Isolation, & Durability) and high-performance querying.

- **Dashboard Development:**

By connecting Microsoft Power BI to Gold Layer via the Azure Synapse Analytics SQL Warehouse, which involves creating measures to visualize the financial provisions and stockout risks.

3.3.5 Testing Phase

The testing phase is crucial when it comes to the accuracy of the financial logic, especially in validating the systems. The process will begin by testing the Synapse Analytics functions and end-to-end integration flow of the system through the Medallion Layer. The error handling testing allows the system to be resilient to data inconsistencies. The dashboard evaluation will be evaluated based on the seven business questions defined to ensure that the Power BI visualization would answer the core business questions.

3.4 Technology Used Description

The project uses Microsoft Power BI and the Microsoft Azure cloud platform to build an end-to-end data engineering pipeline. Azure Data Lake Storage (ADLS) Gen2, which supports the use of Medallion Architecture containing bronze, silver and gold layers storage. Azure Data Factory (ADF) is the orchestration engine that manages data migration and input. Six artificial CSV files are automatically pulled from a Github repository into the bronze layer using HTTP requests configured in ADF. This ensures the raw data is ingested as-is while undergoing first quality checks for duplicates and nulls.

Azure Synapse Analytics processes the data within the Silver layer during the transformation step. The business logic provide by PPG is applied to the raw data using Synapse Analytics which is determining 12-month material consumption to identify Recoverable Assets (RA) and designating materials as Magna Carta (MC) Dead Stock or Expired based on lot expiration dates and consumption history. Computing for dead stock and expired materials is required identifying materials that are at danger of stockout and financial position. The layer makes sure the data has been cleansed, filtered and enhanced with the required business flags.

The pipeline's serving layer is also built using Azure Synapse Analytics which also acts as the gold layer's data warehouse. With the `fact_inventory_status` table at its center the transformed data from the silver layer is organized into a star schema in Synapse. The RA/MC flags and monthly financial values per material are joined in this table that is optimized for

analytical queries. Finally, the star schema from Azure Synapse is linked with MS Power BI to produce interactive dashboards to visualize the results.

3.5 Chapter Summary

Chapter 3 explains the step-by-step plan on how a data engineering pipeline of PPG can be built. The project is based on the Data Engineering Lifecycle which is supported by the System Development Life Cycle (SDLC) that helps to organize the development and achieve business objectives. This methodology is separated into five key stages which are project schedule planning, analysis of business needs, designing data architecture, system implementation with the help of cloud services and testing the results for accuracy. The above phases have proven successful in the project in order to ensure that the raw data can be successfully converted into useful information related to inventory and financial risk.

One of the main architectures in technical design is the Medallion Architecture which structures data into three different layers: Bronze, Silver and Gold. The Bronze layer stores the original raw data, the silver layer is used to clean up and apply logic to identify inventory issues such as expired stocks and lastly the gold layer organizes everything into a star schema format that is ready for reporting. This project includes a set of Microsoft Azure resources that transfer data such as Microsoft Azure Data Factory and a set of resources that perform complex calculations, such as Microsoft Azure Synapse Analytics. The result is an interactive Power BI dashboard that helps PPG gain insight from the data and monitor their inventory's health while protecting customer orders.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter describes the analysis and design of the proposed data engineering pipeline for PPG Inventory Risk Management. It gives a general explanation about the structure of the overall system, requirements, architecture, designing of Database and Dashboard design in the project. This chapter also further explains the proposed system which transforms raw inventory data from the Bronze, Silver and Gold layers to produce information suitable for analytics. Moreover, this chapter provides information on the system requirements, current system analysis, and the design of the end-to-end pipeline with Microsoft Azure services and Power BI. The proposed design is designed to facilitate automated data handling, financial risk assessment and decision making by means of interactive visualization.

4.2 System Analysis

This section analyzes the workflow of the case study in order to understand the business workflow.

4.2.1 Case Study

The case study for this project focuses on the manufacturing and technology operations of PPG's Malaysia plant. PPG manages a complex supply chain involving various distinct raw materials categorized into pigments, resins, solvents, and packaging. Currently, the company faces challenges with inventory obsolescence that leads to financial losses when stock on hand exceeds what the business will realistically use. To solve these issues, the project focuses on two specific risk classifications:

- **Recoverable Assets (RA):** Recoverable Assets refer to raw materials that remain in inventory below 12 months of consumption. Since these materials may no longer be fully

recoverable, they are subject to a formal review process every quarter to assess their status and potential usage.

- **Magna Carta (MC):** Magna Carta represents a higher risk category within Recoverable Assets and requires a complete financial write-down. This category is divided into two groups:
 - a. **Dead Stock:** Materials that have not been consumed for more than 9 months
 - b. **Expired Materials:** Materials that have passed their designated expiry date and can no longer be used.

4.2.2 System Requirements Gathering Techniques

Two main techniques were used to gather the system requirements to ensure the proposed data engineering pipeline meets the business needs of PPG.

1. Document and Data Schema Analysis

The main method employed in this project was the analysis of the project documentation and the six data sets that were provided by PPG. The documents were analysed to explore the relationship between different datasets and each transformation in the data pipeline. Furthermore, 7 business questions were reviewed in the document to identify the metrics, reports and visualisations needed on the visual dashboard. This analysis helped to identify the possible project outputs like data dictionary, data architecture diagram and schema design.

2. Industry Visit and QNA

The interactive question and answer sessions with the PPG representatives and industry speakers, was another important source of information. The discussions highlighted the flow of operations of the business, the data collection from enterprise systems and how the Magna Carta classifications are used in actual situations in the business, and the departments that use inventory reporting for decision making. This ensures that the final outcome of the project matches with their systems.

4.3 System Requirements

System requirements are the functions and quality attributes that must be met for the proposed data engineering pipeline to be effective. In this project, the requirements are split up into

functional and non-functional requirements. Functional requirements are the essential functions the system has to provide, whereas non-functional requirements are those which relate to system quality, performance, scalability and reliability.

4.3.1 Functional Requirements

Functional requirements are the essential functions and operations that the proposed system should be able to perform. For this project, the functional requirements are about the support of the end-to-end data engineering pipeline from data ingestion, transformation, storage, to analytics processing and visualization of the data on a dashboard for inventory risk management, as shown in Table 4.3.1.1.

Table 4.3.1.1 : Functional Requirements

No	Functional Requirement	Description
1	Data Ingestion	The system shall ingest six CSV datasets into the Bronze Layer using Azure Data Factory.
2	Data Cleaning	The system shall remove duplicates, handle null values, and validate data formats.
3	Data Transformation	The system shall apply RA and MC business logic to classify inventory risks.
4	Data Storage	The system shall store processed data in Bronze, Silver, and Gold layers.
5	Star Schema Generation	The system shall organize transformed data into a star schema for analytics.
6	Dashboard Visualization	The system shall generate Power BI dashboards to answer business questions.
7	Stockout Risk Detection	The system shall identify materials with stockout risk and affected sales orders.

4.3.2 Non-Functional Requirements

Non-functional requirements are those requirements that specify the quality attributes and performance criteria of the proposed system. These requirements guarantee efficient, reliable, and secure data engineering pipelines, ensuring scalability and maintainability for future improvements, as shown in Table 4.3.2.1.

Table 4.3.2.1 : Non-Functional Requirements

No	Non-Functional Requirements	Description
1	Scalability	The system shall support increasing data volume efficiently.
2	Reliability	The system shall ensure consistent and accurate data processing.
3	Performance	The pipeline shall process data within acceptable execution time.
4	Data Accuracy	The system shall maintain accurate RA and MC calculations.
5	Availability	The dashboard shall be accessible for monitoring and reporting purposes.
6	Maintainability	The pipeline shall allow future modification and enhancement.
7	Security	Access to cloud storage and services shall be managed securely through Microsoft Azure.

4.4 Current System Analysis

The current system is analyzed to determine the current workflow to identify problems or bottlenecks that need to be addressed. This helps in a more targeted approach addressing problems that are significantly reducing the efficiency of the workflow so that the proposed solution better resolves those issues.

4.4.1 Current System and The Bottlenecks

In this section, some of the reasons for using a new data pipeline was explained by going through the current workflow at the PPG. The system currently consists of manual processes and static spreadsheets. Each department is operating on separate databases with operational tasks done using different software tools resulting in massive data silos.

Since there is no central system to connect these different areas, the PPG's operational records are saved as six individual, disconnected CSV files which are materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv and sales_orders.csv. When the team requires to perform the inventory risk calculation or follow up on material safety stock, it requires an analyst to collect these files and copy and paste the data to other sheets.

The existing configuration presents the following challenges:

- a. **Time-Consuming Risk Classification:** What is currently taking a lot of time is the effort of manually writing and running a complex lookup formula in a spreadsheet to sort materials and look for all Recoverable Assets (RA) or raw materials that have been in stock for more than 12 months of consumption. The same tiring row-by-row checking happens when applying the Magna Carta (MC) rules to flag Dead Stock, which are the items with zero consumption for 9 months or identifying expired stocks.
- b. **Slow and Error-Prone Financial Estimates:** Financial risk numbers are calculated using manual spreadsheets which makes the process of determining all financial write-down provisions for the finance team slow. If one of the formulas is wrong or one analyst types the wrong one, the reports given to the management team about the financial risks are entirely incorrect.
- c. **No Communication Between Files:** The six main CSV data sheets reside in totally different folders and do not communicate to each other automatically. For instance, if the procurement team is notified of a delay in delivery, the details are entered into purchase_orders.csv but it does not automatically update the stock levels or material categories.
- d. **Reactive Troubleshooting for Customer Orders:** Another challenge is that the sales team does not have visibility on how the level of stock is aligned with supplier lead times when handling customer orders. Once a raw material is exhausted, there is no quick solution in

response to the shortage for customer ordering. The team usually only finds out about a problem after a customer delivery fails.

4.4.2 Flow Chart for the Current Process

The current system works sequentially and step by step and involves a lot of manual intervention as shown in Figure 4.4.2.1. It starts with all the 6 siloes data generated from the excel files from different departments. They all generate individual and separate data logs without a central log point to control them. Thus, it is important that there is an inventory analyst to cover these space gaps for administration. They have to spend key working hours collecting, tidying and cross-referencing rows from various spreadsheets. After merging the data, the data analyst has to implement the main business logic, checking the lines one by one. They verify if a material's current stock exceeds 12-month of expected consumption baseline, classify it as a Recoverable Asset (RA) or check if an item has suffered from 9 months of total inactivity or expired, flag it as a Magna Carta (MC) risk item. Once these checks are completed the analyst will calculate the financial reserves by putting them together on a spreadsheet and pass it to the finance team. They need to manually update the financial write down provisions by 100%. Last but not least, the cycle is affected when faced with stock disruptions with the current customer order pipeline being fully separated from the real-time inventory system. The supply chain team only can act on a stock out of raw material once a customer delivery has been in arrears for delivery. By that time, it is already too late for PPG to solve this issue.

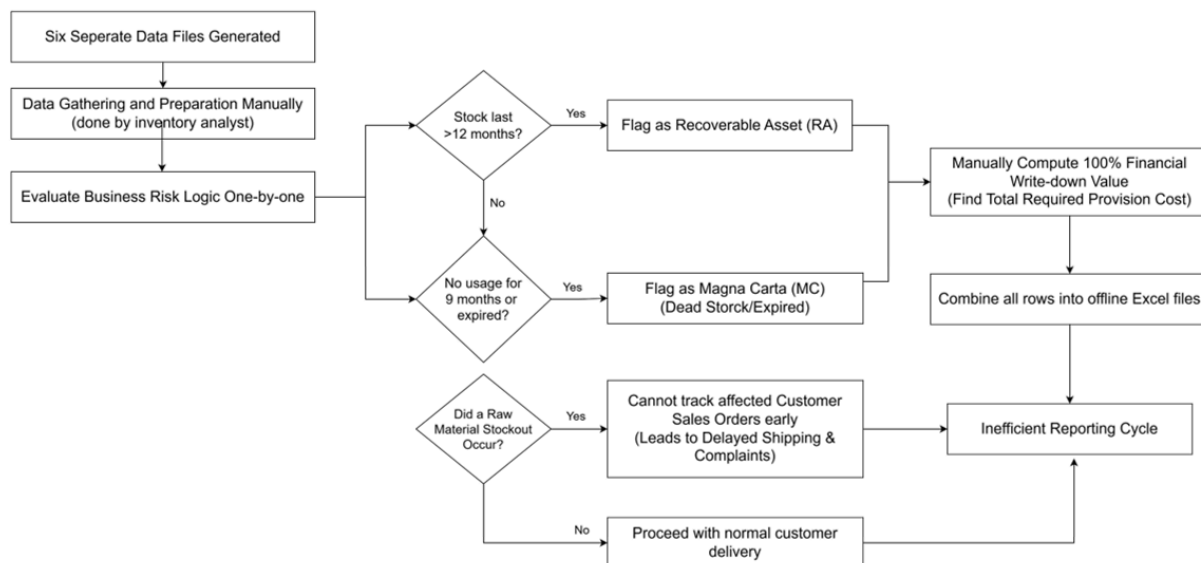


Figure 4.4.2.1 Current Manual Inventory Risk Management Flow Chart

4.4.3 Key Improvements

To solve these issues, an end-to-end automated cloud data pipeline is designed to completely replace this manual spreadsheet method:

- a. **Automated Data Ingestion:** Azure Data Factory will be used to automatically pull all six CSV files into a centralized cloud data lake (Bronze layer) to prevent the need to open and copy files manually.
- b. **Code-Based Automated Cleaning:** Instead of using Excel formulas, Azure Synapse notebooks are used to clean data, handle null values, remove duplicates and calculate the RA and Magna Carta logic instantly and reliably.
- c. **Centralized Data Modeling:** All the cleaned and transformed information will be structured into a Star Schema inside our Gold layer data warehouse to make sure data retrieval is extremely fast.
- d. **Proactive Dashboard Insights:** A live Power BI dashboard connected directly to our data lakehouse serving layer will be built. This allows managers across different departments to check the stockout risks, financial write-downs and affected customer orders immediately to make smart decisions quickly before problems happen.

4.5 System Design

This section breaks down the complete system design for PPG's Inventory Risk Management data pipeline. It covers the overall architecture and explains exactly what each Azure component does, followed by a detailed look at the data engineering process itself. Finally, it outlines the database design including the star schema and normalized tables as well as the interface design for the Power BI dashboard.

4.5.1 System Architecture

This section breaks down the system design for the PPG Inventory Risk Management data pipeline. It evaluates the overall architecture and how each Azure component fits together, followed by a look at the data engineering workflows. Finally, the technical specifications

outline the database design including the star schema and normalized tables as well as the interface layout for the Power BI dashboards.

The overall system is built on four Azure services, Azure Data Factory (ADF), Azure Data Lake Storage Gen2 (ADLS Gen2) and Azure Synapse Analytics, with Microsoft Power BI as the final reporting interface. Each service occupies a specific role in the pipeline and passes data forward to the next component. Figure 4.5.1.1 below shows the full end-to-end system architecture, illustrating how data flows from the six CSV source files through the Bronze, Silver and Gold layers into the Power BI dashboard.

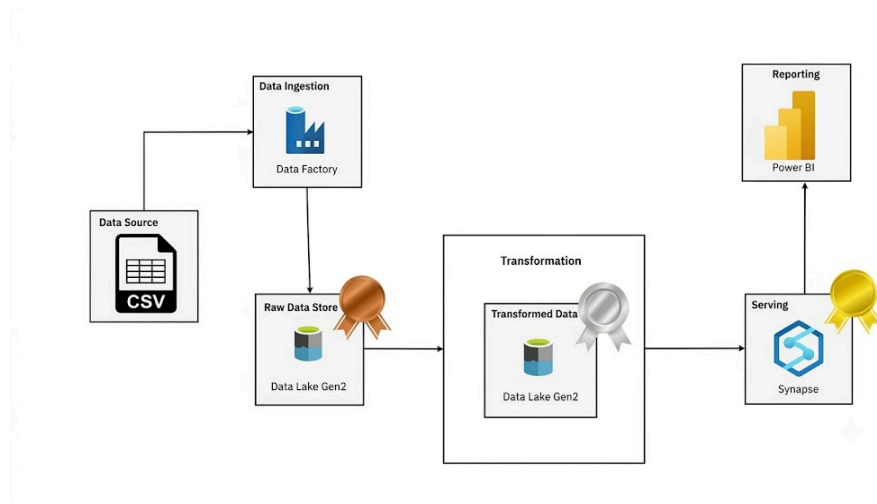


Figure 4.5.1.1 : End-to-end System Architecture

The pipeline begins with the six synthetic CSV source files PPG's operational records which are metrials.csv, inventory_snapshot.csv, purchase_orders.csv, productiojn_orders.csv, suppliers.csv and sales_orders.csv. These files are automatically extracted and loaded into the Bronze Layer of ADLS Gen2 by ADF, preserving all raw data without modification. From the Bronze Layer, Azure Synapse Analytics reads the raw files, applies data cleaning and the full set of RA and MC business logic transformations and writes the enriched output into the Silver Layer of ALDS Gen2 as Parquet files. The processed Silver data is then loaded into Azure Synapse Analytics, where it is structured into a Star Schema to form the Gold Layer. Finally, Microsoft Power BI connects to the Synapse Gold Layer via a live SQL connection and renders the interactive dashboard that answers the seven defined business questions (Q1-Q7). Figure 4.5.1.2 below summarises this data flow across the three Medallion Layers.

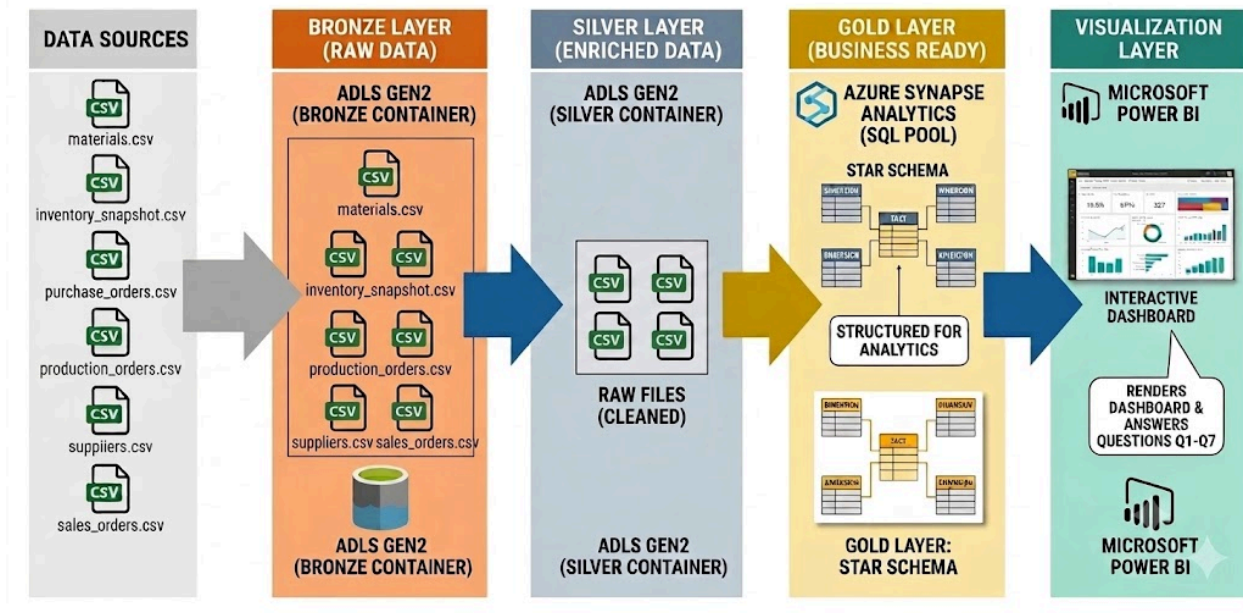


Figure 4.5.1.2 : System Medallion Layer Data Flow Architecture

This architecture was designed so that each layer has a single, well-defined responsibility. The Bronze Layer is concerned only with reliable ingestion and raw storage. The Silver Layer is concerned only with transformation and business logic. The Gold Layer is concerned only with serving clean, query-optimised data for reporting. This separation ensures that any layer needs to be modified.

4.5.2 Component In System Architecture

This section breaks down the configuration and operational scope of each Azure component within the pipeline, outlining how individual cloud services integrate to maintain continuous data flow between processing stages.

- (a) **Azure Data Factory (ADF)** handles all the centralized orchestration, using six dedicated ingestion pipelines to automate data movement. Run by scheduled triggers, each pipeline uses a Copy Data activity to pull source files directly into their assigned folders within the ADLS Gen2 Bronze Layer. During the transfer, the system runs basic schema-level quality checks to verify data types and catch any missing values in critical identifier

fields. For continuous oversight, ADF's monitoring dashboard archives all pipeline execution logs, making it easy to maintain a complete ingestion audit trail.

(b) Azure Data Lake Storage Gen2 (ADLS Gen2) acts as the central repository, hosting the three separate containers needed for the Medallion Architecture. The Bronze container stores the six raw CSV files exactly as they arrived, giving us an unedited source of truth for data lineage. The Silver container holds the clean, optimized Parquet files generated by Azure Synapse Analytics after running the core business logic. Finally, the Gold container stores the final, analytics-ready datasets. Security and data isolation across these tiers are handled through Azure Role-Based Access Control (RBAC), which restricts each pipeline service to write data only within its designated target container.

(c) Azure Synapse Analytics Notebook the transformation engine for the Silver Layer by using Apache Spark. Synapse Analytics notebook are written in PySpark to carry out the following tasks in sequence:

1. Removing duplicate records and handling null values across all six datasets
2. Calculating 12-month rolling material consumption to identify Recoverable Assets (RA)
3. Flagging MC Dead Stock for materials with zero consumption over nine or more consecutive months
4. Flagging MC Expired materials where the lot expiry date in `inventory_csv` has already passed
5. Calculating 100% financial write-down provision amounts for all MC-classified materials
6. Comparing current stock levels against supplier lead times to identify stockout risks
7. Mapping at-risk inventory items directly to open sales orders to evaluate consumer fulfillment risks

(d) Azure Synapse Analytics hosts the Gold Layer warehouse, where the data is structured into a star schema based on the design criteria from Section 2.2.4. The layout centers on

the fact_inventory_status table, which tracks historical monthly snapshots for each material along with calculated RA and MC flags, financial provisions, and stockout metrics. This central fact table connects directly to four dimension tables: dim_material, dim_supplier, dim_date, and dim_sales_order. This setup makes it easy to slice and dice inventory risks from multiple angles. Finally, the data warehouse is exposed to Power BI via a serverless SQL pool live connection to ensure fast, interactive reporting.

- (e) **Microsoft Power BI** serves as the presentation layer, connecting directly to the Azure Synapse Gold Layer. The resulting dashboard uses targeted visuals to answer all seven key business questions, using interactive slicers so stakeholders can easily filter data by category, time, supplier, or risk profile. Once completed, the report is deployed to the Power BI Service, where role-based permissions are used to control data visibility and ensure proper departmental security.

4.5.3 System Design

This section will outline the blueprint of the proposed data engineering solution where the details of the data can be transformed into an optimized analytical model by implementing the core business logic that is required from the industry.

4.5.3.1 Data Engineering Process

The process is designed as a Medallion Architecture that is implemented within Microsoft Azure where each layer has its specific role.

- **Bronze Layer:** Creates an unchangeable historical ledger where the data from the source files is ingested with zero schema modifications by preserving the original system timestamps, string types, and raw structural layouts to facilitate full operational auditability.
- **Silver Layer:** This layer will represent the business computing boundary where data cleansing and enterprise logic will be applied to the data in a programmatic manner. The Silver layer has been designed to process parallel computing chains which are then

programmatically joined into a single comprehensive dataset before being sent to the warehouse.

- **Gold Layer:** Responsible for structuring the analytics ready for presentation where the tables will be denormalized from the combined Silver layer and written into Azure Synapse Analytics. The architecture will shift the structural layouts into a dedicated Star Schema by creating optimal primary/foreign key connections that eliminate complex recursive join runtime and instantly rendering using DirectQuery or Import for Microsoft Power BI.

4.5.3.2 Data Source

The six synthetic datasets will be ingested by the pipeline that mirror the transactional entities generated across the plant's operational departments. The granular configuration of these data sources is defined in Table 4.5.3.2.1.

Table 4.5.3.2.1 : Data Source Dictionary and Core Mapping

Dataset Source File	Primary Key	Core Mapping
materials.csv	material_id	Maps the raw chemical components across categories
inventory_snapshot.csv	snapshot_date	Evaluates the active stock values and lot-expiry evaluations
purchase_orders.csv	po_id	Quantify the incoming open supplier lines to calculate the true net stock availability
production_orders.csv	production_order_id	Records the history usage details across batches and create a baseline demand values
suppliers.csv	supplier_id	Integrates the supplier metrics into the reporting model
sales_order.csv	sales_order_id	Maps customer demand patterns to trace specific customer order shortages

4.5.3.3 Extraction, Transformation, and Loading (ETL)

The data movement pipeline is optimized by Extract-Load-Transform-Load (ELTL) sequence by using Azure Data Factory (ADF) for system orchestration, Azure Synapse Apache Spark Pool (sparkpool) for scalable multi-node processing, and an Azure Synapse Analytics Serverless SQL Pool for analytical hosting.

1. Extraction Phase

This phase is controlled by Azure Data Factory through explicit linked services that handle six automated, non-blocking parallel Copy Data workers that extract raw transaction files and stream the data directly into the directories without changing the metadata formatting. In order to safeguard the warehouse against downstream processing errors, a mandatory data flow gate checks for structural anomalies where it isolates records into two functional channels through a strict conditional split:

- **Null Field Imputation:** Rows containing empty identifiers or quantitative metrics are identified and routed to a separate directory. The non-critical missing components, such as undefined lot expiration dates, are programmatically mapped to a distant fallback token (9999-12-31) to ensure calculation compatibility.
- **Deduplication Loop:** Records are directed to different directories through an automatic aggregation node based on explicit natural keys.

2. Transformation Phase

In this phase the data transformation is executed within Synapse Notebooks utilizing an active PySpark framework. The transformation layer uses a two-pronged script structure to execute inventory equations before merging the datasets:

- **Risk Metric Calculations:**

Evaluates asset recovery indicators and financial provisions by querying historical production order datasets against live inventory lines:

Recoverable Assets (RA): Computes cumulative material consumption over a rolling 12-month window ($C_{m(12)}$). If the current warehouse quantity on hand (I_m) exceeds that consumption baseline, the material is classified as a Recoverable Asset, and its absolute

volume surplus is quantified:

$$\text{If } I_m > C_{m(12)} \rightarrow is_recoverable_asset = True, excess_quantity = I_m - C_{m(12)}$$

Magna Carta (MC) Dead Stock: Identifies total material inactivity. If a material records zero production consumption over a consecutive 9-month historical period ($C_{m(9)} = 0$) and its lot expiration column is null, it is flagged as Dead Stock:

$$\text{If } (\sum_{t=-9}^{current} C_{m,t} = 0) \wedge (lot_expiry_date \text{ is Null}) \rightarrow is_mc_dead_stock = True$$

Magna Carta (MC) Expired: Compares the system runtime execution date ($D_{current}$) against batch expiration dates (D_{expiry}). When a batch exceeds its expiration date, it triggers a 100% loss provision using the material's standard cost:

$$\text{If } D_{current} \geq D_{expiry} \rightarrow is_mc_expired = True, mc_provision_amount = I_m \times unit_cost$$

- **Stockout Analysis & Explicit Column Ownership:**

This script evaluates operational supply chain buffers and customer impact where it resolve ownership ambiguity, this block explicitly calculates and controls the `is_below_reorder` indicator:

Reorder Level Checking: Generates the reorder column by matching current balances directly against safety limits:

$$is_below_reorder = I_m < reorder_level$$

Stock Cover Mapping: Calculate the average daily consumption rates (ADC_m) across historical production dates. The pipeline calculates how many days of operational stock cover remain (SC_m) and compares it against supplier lead times (LT_m) to detect supply risks:

$$ADC_m = \frac{Total\ Quantity\ Consumed}{Active\ Production\ Days\ Range}$$

$$SC_m = \frac{I_m}{ADC_m}$$

$$\text{If } SC_m < LT_m \rightarrow is_stockout_risk = True$$

Customer Order Alignment: Maps open sales orders against at-risk raw materials by cleaning string fields through the join key which create a clear customer risk visibility pipeline.

- **The Merged Data Consolidation:** In order to fix the data loss issue, a final processing step joins the output that unifies a transactional Parquet table inside the Silver directory.

3. Loading Phase

The merged silver rows are committed directly to disk using the Delta Lake framework, ensuring transactional ACID (Atomicity, Consistency, Isolation, Durability) is compliant across the multi-node execution paths. The master orchestration pipeline will trigger an automated SQL to launch the update process that uses optimized operations to read from the consolidated Silver Parquet location and upsert data directly into the Azure Synapse Gold analytics repository. The end results will create three main dimensions tables and an autogenerated calendar table that circulate around a central fact table.

4.5.3.4 System Process Design

This section presents the system process design of the Inventory Risk Management and Recoverable Assets Dashboard using three modelling diagrams which consists of Use Case Diagram, Sequence Diagram, and Activity Diagram.

Use Case Diagram

The Use Case Diagram in *Figure 4.5.3.4.1* identifies the primary actors and their interactions with the proposed system. Actors identified are Supply Chain Manager, Inventory Manager and Data Engineer.

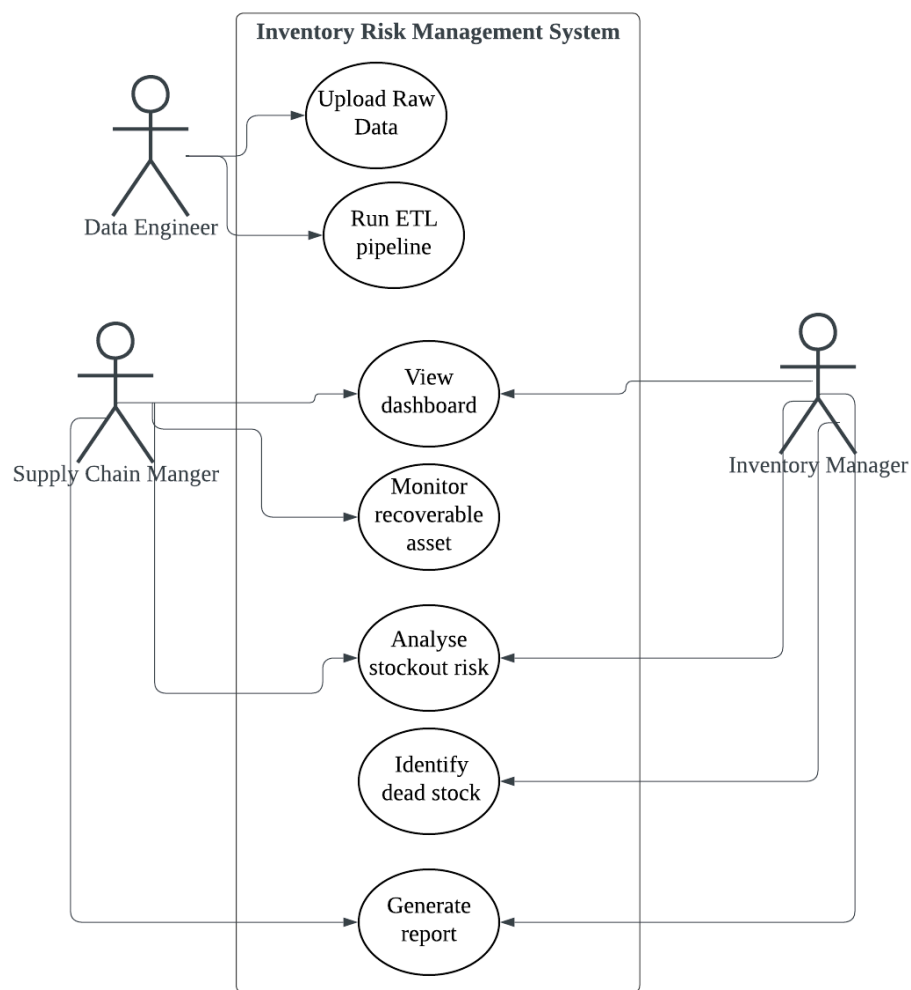


Figure 4.5.3.4.1 : Use Case Diagram

Activity Diagram

The Activity Diagram models in *Figure 4.5.3.4.2* shows the end-to-end workflow of the system process from data ingestion to dashboard consumption.

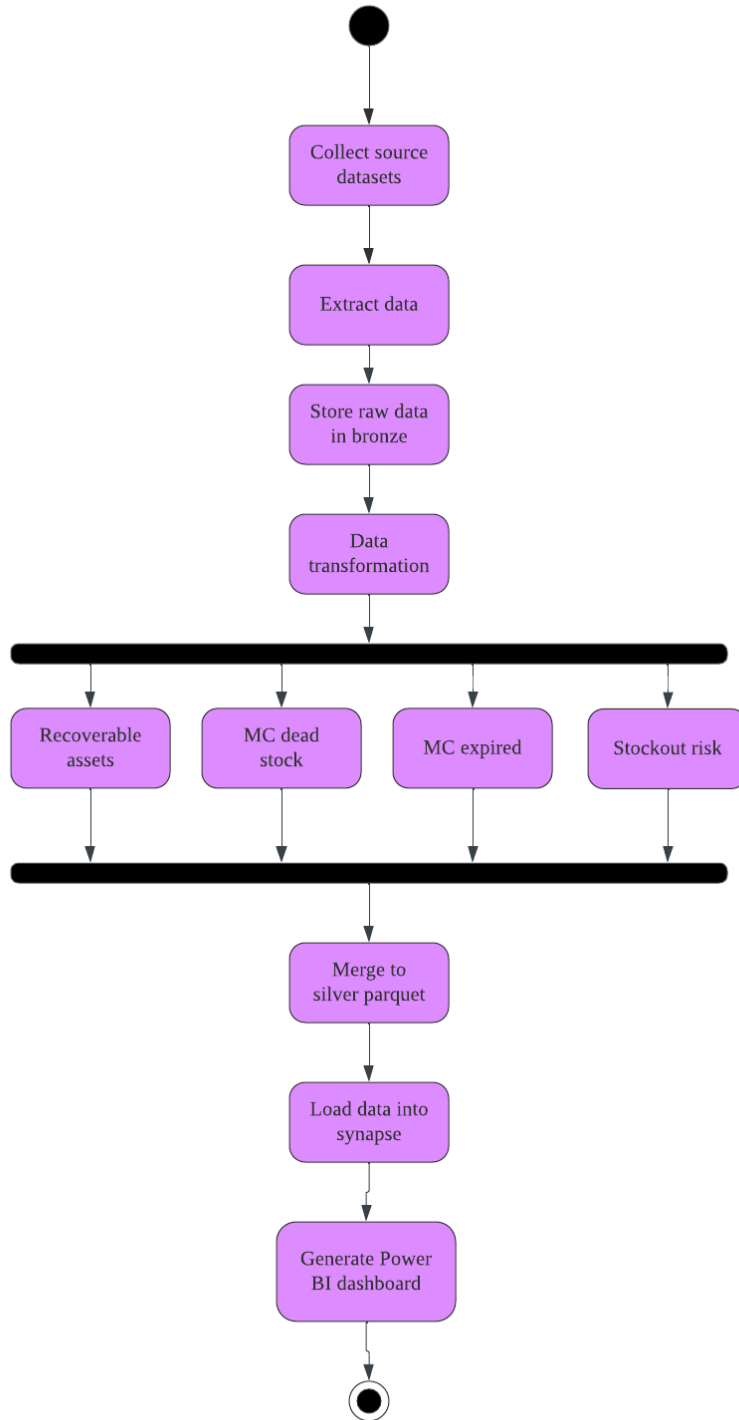


Figure 4.5.3.4.2 : Activity Diagram

Sequence Diagram

The Sequence Diagram in *Figure 4.5.3.4.3* illustrates the flow of interactions between system components during the execution of the data pipeline and dashboard rendering.

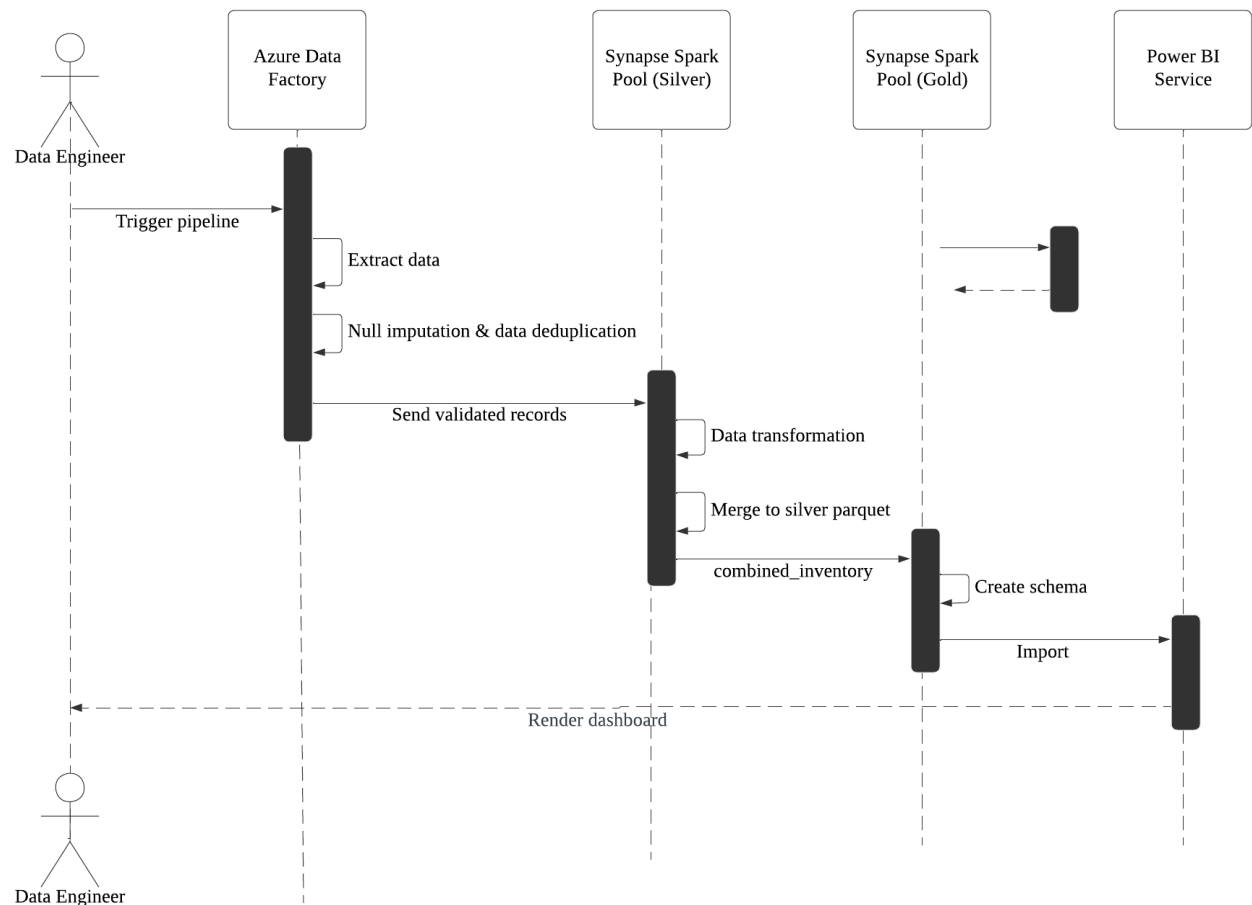


Figure 4.5.3.4.3 : Sequence Diagram

4.5.4 Database Design

The database design for the Recoverable Assets and Inventory Risk Management system follows a two-layer modelling approach, reflecting the medallion architecture adopted by the pipeline. The bronze and silver layer preserves the structure of the source system and is represented by an Entity Relationship Diagram (ERD) normalised to Third Normal Form (3NF), ensuring data integrity and eliminating redundancy. The analytical layer in the gold and curated layer is represented by a multidimensional star schema optimised for the seven business questions identified in Section 4.5.2, where query performance and analytical clarity outweigh storage efficiency. Together, these two models cover both the transactional and analytical responsibilities of the system.

4.5.4.1 Entity Relationship Diagram (ERD)

The ERD models the operational source layer of the pipeline, capturing the entities and relationships that exist in PPG Industries' raw material, procurement, production, and sales records. It serves as the logical foundation for the Bronze and Silver layers of the data lakehouse, where data is ingested and cleansed before any business rules are applied. The database design can be shown in Figure 4.5.4.1.1 below.

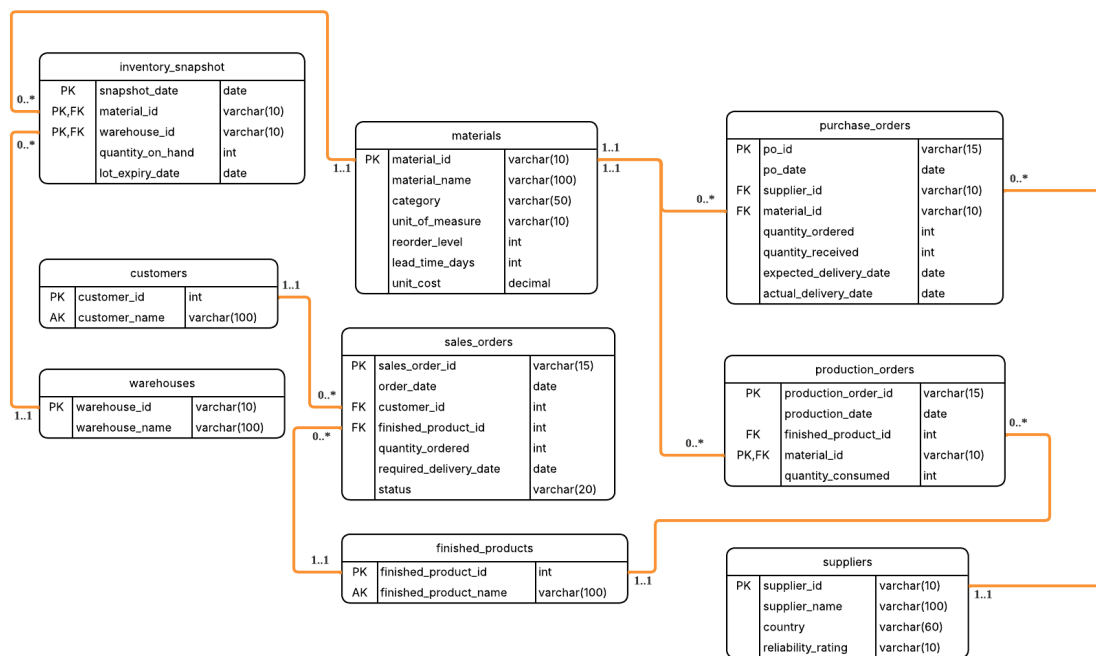


Figure 4.5.4.1.1 Normalized ERD of PPG Inventory Management System

Entities and Descriptions

Table 4.5.4.1.1 shows the entities involved in the ERD with respective primary keys and composite keys involved with description for each table.

Table 4.5.4.1.1 Entities and Description of ERD

Entity	Primary Key	Description
materials	material_id	Catalogue of 20 raw materials across four categories (Pigment, Resin, Solvent, Packaging). Stores reorder thresholds, lead times, and unit costs that drive RA/MC classification.
suppliers	supplier_id	Supplier reference data including country and reliability rating.
warehouse	warehouse_id	Storage locations holding inventory. Extracted to remove string repetition in inventory_snapshot.
customers	customer_id	Customers placing sales orders. Extracted to support customer-level traceability of stockout impact.
finished_products	finished_product_id	Finished paint and coating products. Extracted to provide the structural link between production output and customer demand.
inventory_snapshot	snapshot_date, material_id, warehouse_id	Monthly stock-on-hand levels per material per warehouse. Composite key reflects the grain: the same material may be stored across multiple warehouses on the same snapshot date.
purchase_orders	po_id	Purchase orders with planned and actual delivery dates, supporting analysis of late or partial deliveries.
production_orders	production_order_id, material_id	Bill-of-materials consumption records. Composite key reflects the fact that a single production run consumes multiple raw materials.
sales_orders	sales_order_id	Customer demand for finished products, including status (Open, Delivered, Cancelled).

Relationships and Cardinality of ERD

The relationships between entities using UML notations for cardinality is shown in Table 4.5.4.1.2 below.

Table 4.5.4.1.2 Entity Relationships and Cardinality of ERD

Primary Entity	Relationship Type	Related Entity
materials	1..1 — 0..*	inventory_snapshot
materials	1..1 — 0..*	purchase_orders
materials	1..1 — 0..*	production_orders
suppliers	1..1 — 0..*	purchase_orders
warehouses	1..1 — 0..*	inventory_snapshot
customers	1..1 — 0..*	sales_orders
finished_products	1..1 — 0..*	sales_orders

4.5.4.2 Multidimensional Modelling

A multidimensional model is used for answering the seven business questions as particularly those involving aggregations across millions of inventory snapshots, multi-table joins, and pre-computed risk flags requires a model designed for read-heavy access patterns. The Gold and Curated layers therefore adopt a star schema shown in Figure 4.5.4.2.1

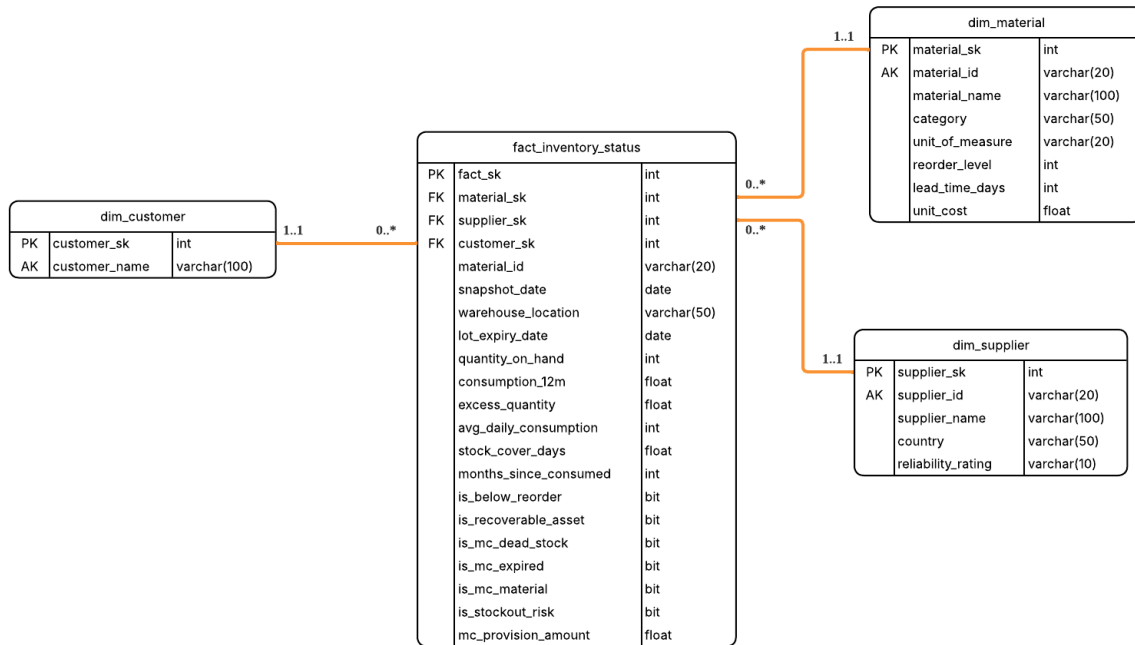


Figure 4.5.4.2.1 Star Scheme for Inventory Dashboard System

Fact Table

The fact_inventory_status is the central fact table of the schema. It carries three categories of columns with foreign keys to dimensions which are material_sk, supplier_sk and customer_sk. It replaces the natural business keys present in the source system, isolating the analytical layer from any future changes in source key formats and improving join performance through narrow integer keys. The fact table consists of numeric measures which are quantity_on_hand, consumption_12m, excess_quantity, mc_provision_amount, stock_cover_days, avg_daily_consumption, months_since_consumed and boolean classification flags which are is_below_reorder, is_recoverable_asset, is_mc_dead_stock, is_mc_expired, is_mc_material and is_stockout_risk. The flags encode the business rules and are evaluated once during the Gold transformation, so downstream queries reduce to simple filters.

4.5.4.2 Dimensions Table

The dimensions tables are used in analysis using functions such as filters and joins to provide further description on the aggregated data. Table 4.5.4.2.1 shows the dimensions table in the start schema with its description.

Table 4.5.4.2.1 Dimensions Table in Star Scheme and Description

Dimension	Foreign Key	Primary Key	Description
dim_material	material_sk	material_id	Raw material attributes including category, unit of measure, reorder level, lead time, and unit cost. The unit cost in particular feeds the financial provision calculation.
dim_supplier	supplier_sk	supplier_id	Supplier attributes including country and reliability rating, supporting supplier-level risk analysis.

dim_customer	customer_sk	customer_name	Customers whose open sales orders are at risk due to material shortages, enabling Q7 traceability at the customer level.
--------------	-------------	---------------	--

Relationship and Cardinality

The relationships between fact and dimension tables using UML notations for cardinality is shown in Table 4.5.4.2.2 below.

Table 4.5.4.2.2 Star Scheme Relationship and Cardinality

Parent (Dimension)	Cardinality	Child (Fact)
dim_material	1..1 — 0..*	fact_inventory_status
dim_supplier	1..1 — 0..*	fact_inventory_status
dim_customer	0..1 — 0..*	fact_inventory_status

The database design adopts a deliberate separation of concerns between the operational and analytical layers. The 3NF ERD provides a clean, integrity-preserving foundation for ingesting and cleansing source data, while the star schema delivers a query-optimised, suitable for business model for analytical consumption.

4.5.5 Interface Design

Figure 4.5.5.1 helps the warehouse team check day-to-day stock levels. On the top left, two quick summary cards show the total stock value and the number of items running low. While the bar chart presents which materials are close to running out compared to their reorder safety lines. On the right side, the donut chart and data table split all inventory into three risk categories, which are Healthy, Recoverable Asset and Stockout Risk.

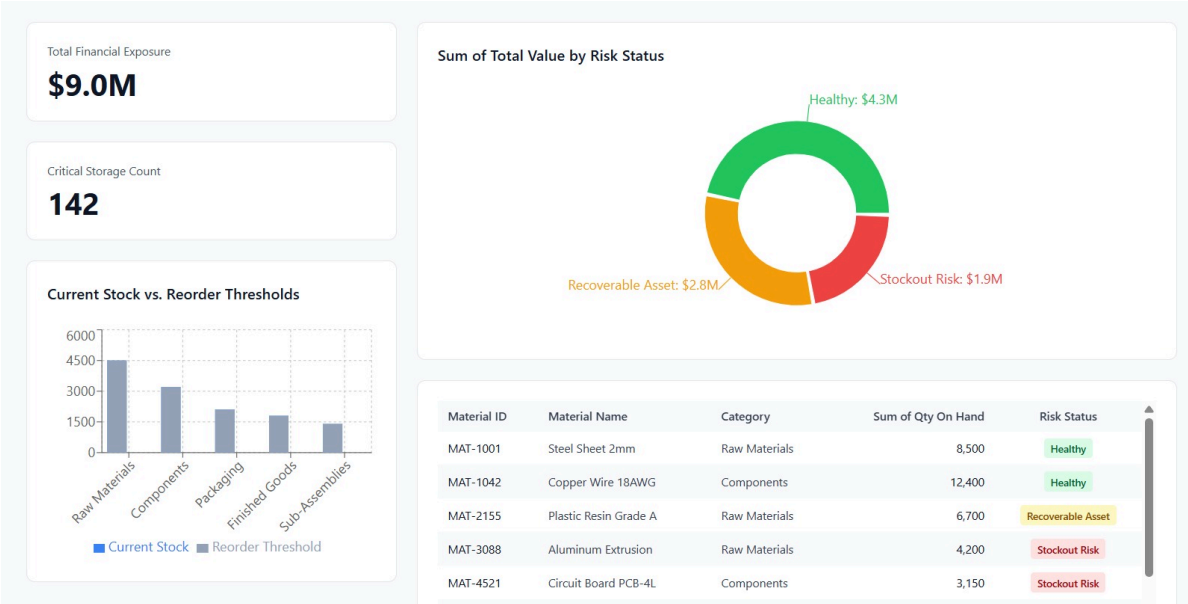


Figure 4.5.5.1 Interface Design for Asset Recovery and Active Inventory Status Dashboard

Figure 4.5.5.2 illustrates the dashboard that acts as an early warning system for late customer deliveries and highlights the material categories that are nearly out of stock. A prominent card created at the top emphasizes the total financial loss from items that were wasted. The donut chart then breaks the loss into two types, which are stock that has been sitting dead for over 9 months (MC Dead Stock) and stock that has passed its expiration data (MC Expired). The table lists the exact materials, their categories, how much money is lost and the MC type.

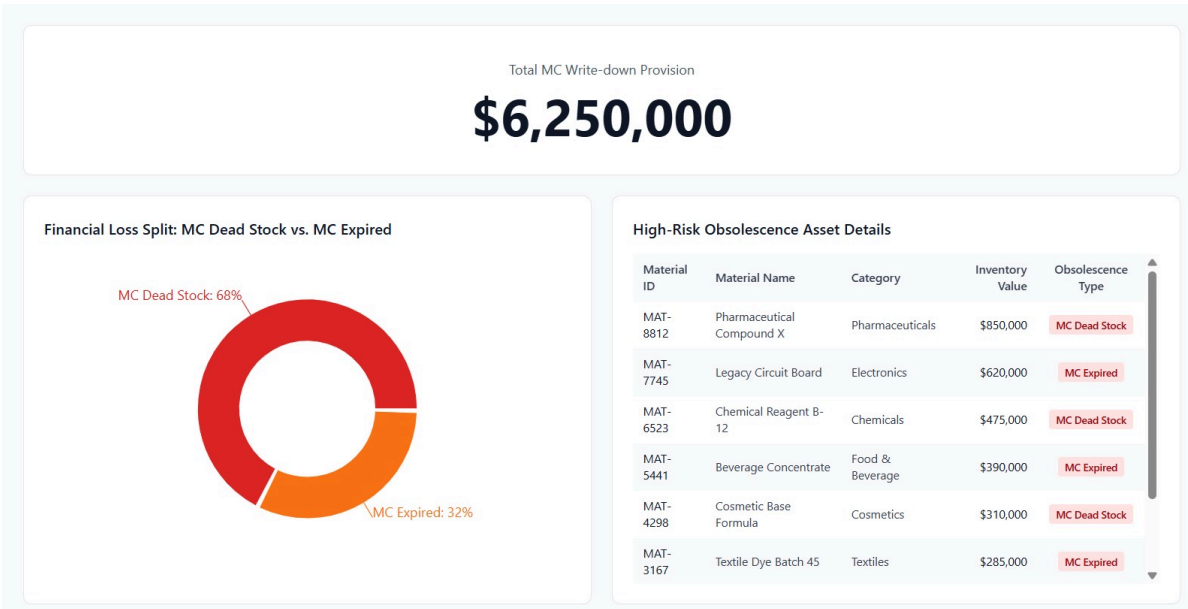


Figure 4.5.5.2 Interface Design for Magna Carta (MC) Financial Provisions Dashboard

This prototype dashboard as shown in Figure 4.5.5.3 acts as an early warning for late customer deliveries and the material category that is out of stock. The bar chart shows which material categories are causing the most delivery issues and may affect most customers. The main table on the right shows the exact customer name and sales order ID that have possibly been affected and will be late, matching how many days of stock we have left against how many days it takes for a new supplier shipment to arrive.

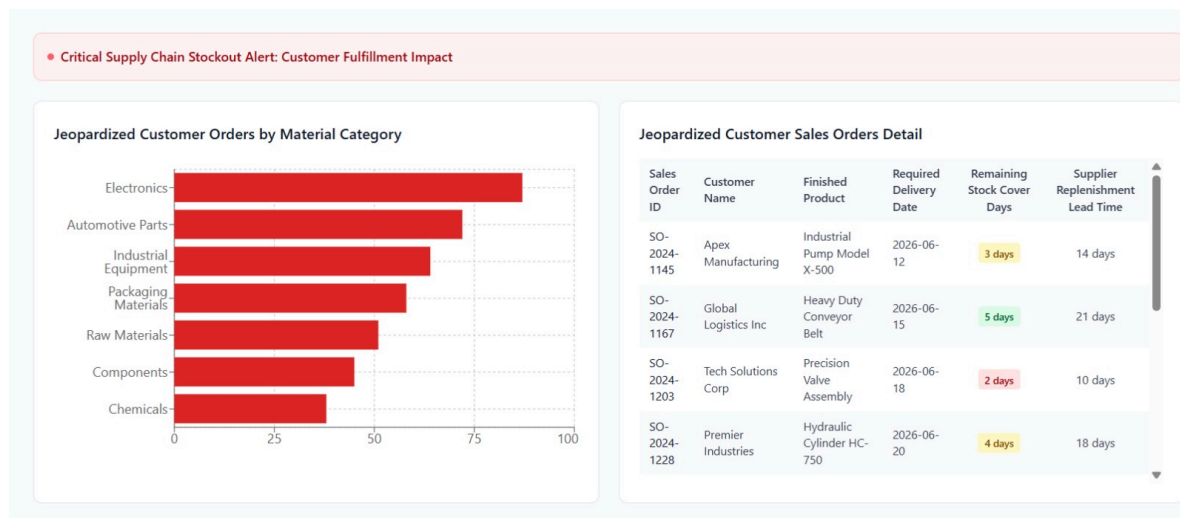


Figure 4.5.5.3 Interface Design for Customer Order Delivery Risks Dashboard

4.6 Summary

This chapter presented the analysis and design of the proposed data engineering pipeline for PPG Recoverable Assets and Inventory Risk Management. The current inventory management process was analysed and several key problems were identified, including disconnected data sources, heavy reliance on manual spreadsheet processing, delayed risk detection and reactive handling of stockout issues. These limitations show that an integrated data pipeline was needed to improve the inventory visibility, financial risk monitoring and for business decision-making.

Functional requirements were focused on data ingestion, data cleaning, transformation, inventory risk classification, financial provision calculation and dashboard reporting. Meanwhile,

non-functional requirements such as accuracy, reliability, scalability, performance and usability were considered also to ensure that the proposed system could support business operations effectively.

The proposed system design was developed using the Medallion Architecture, which consists of Bronze, Silver and Gold layers. Raw data from the six CSV files is first ingested into the Bronze layer, then cleaned and transformed in the Silver layer. Before being structured into an analytics-ready format in the Gold layer. Microsoft Azure services, including Azure Data Factory, Azure Data Lake Storage Gen2 and Azure Synapse Analytics were used to support the data pipeline while Microsoft Power BI was selected as the visualization tool.

In addition, the database design was explained through both a normalized ERD and a star schema. The ERD was used to represent the operational data structure while the star schema was designed to support efficient reporting and analytical queries. The interface design was also visualized using three Power BI dashboard pages which focus on inventory risk overview, Magna Carta financial provisions and customer order delivery risks. Overall, this chapter provided a clear design foundation for implementing an automated, scalable and data-driven inventory risk management system for PPG.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter describes the implementation and testing of the proposed PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) system. The implementation phase is dedicated to building an end-to-end data engineering pipeline that utilizes Microsoft Azure services and Microsoft Power BI to help with inventory risk analysis and business decision making.

The system was deployed following the Medallion Architecture of Bronze, Silver and Gold layers. Azure Data Factory was utilized for data ingestion and data quality processing, Azure Data Lake Storage Gen2 was utilized for data storage, Azure Synapse Analytics was used for data transformation, implementation of business rules, and analytical processing through Synapse Spark Notebooks and SQL scripts. Power BI was used for dashboard visualization and reporting. The system architecture and implementation information, dashboard interfaces and test results are also presented in this chapter.

5.2 System Development / Architecture

The system architecture proposed in Chapter 4 was implemented successfully on Microsoft Azure cloud services and the Microsoft Power BI platform. The implementation is based on the Medallion Architecture that is a structured and scalable data engineering pipeline which consists of the Bronze, Silver, and Gold layers. The architecture supports the systematic ingestion, validation, transformation and preparation of data for analytical reporting.

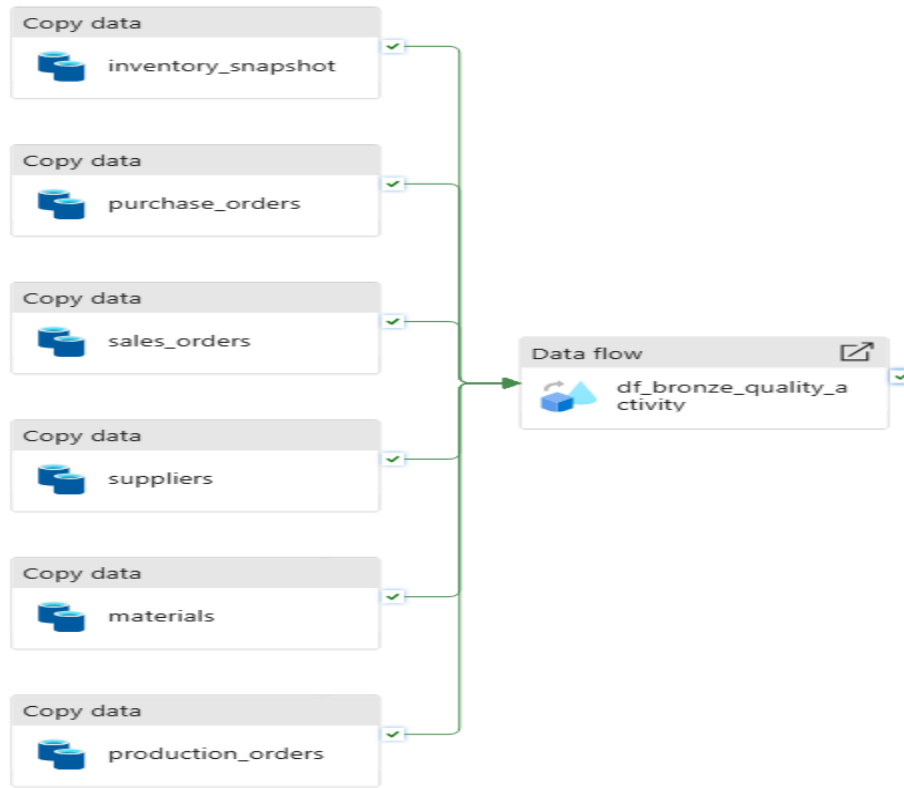


Figure 5.2.1 Azure Data Factory pipeline for extracting raw CSV files into the Bronze Layer

The implementation begins with six operational datasets, namely materials, inventory_snapshot, production_orders, purchase_orders, sales_orders, and suppliers. As shown in Figure 5.2.1, the automated pipeline utilizes Copy Data activities to extract the six operational datasets from the source and load them into Azure Data Lake Storage Gen2 via Azure Data Factory. The Bronze layer is the first layer of storage to hold raw data and perform processing. Azure Data Factory Mapping Data Flows were used to validate data quality and detect data duplicates. Records which passed validation were placed in validated folders and records that were invalid and/or a duplicate were placed in quarantine folders for monitoring and traceability.

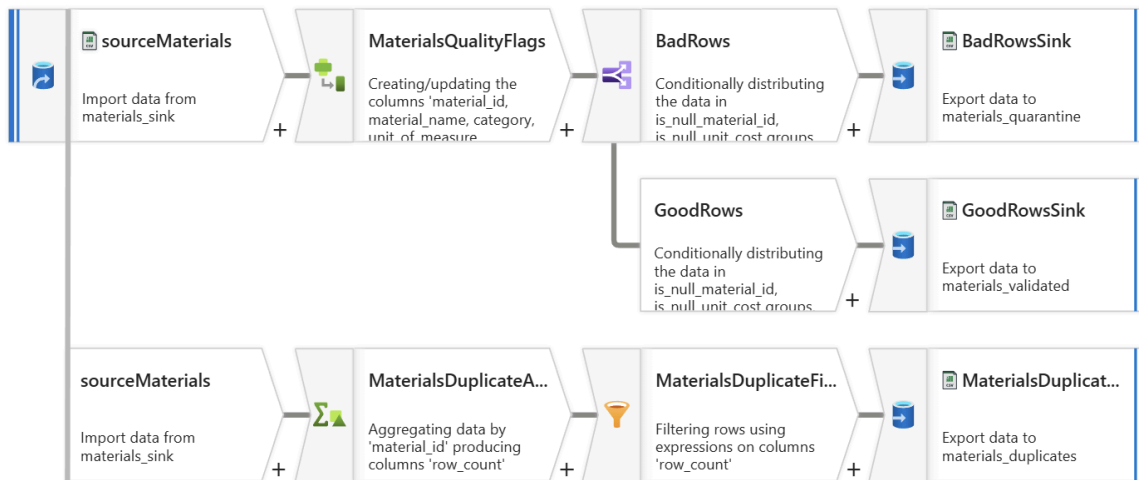


Figure 5.2.2 Azure Data Factory Mapping Data Flow for Data Quality Validation and Duplicate Detection

Figure 5.2.2 shows the Azure Data Factory Mapping Data Flow that is used to validate data quality and detect data duplication in the Bronze layer. The data flow fetches records from the ingested datasets and applies validation rules to detect data records that have missing mandatory values, invalid data formats, or duplicate entries. Valid and invalid records are divided using conditional split transformations, depending on the defined business rules. Records that meet all the validation criteria are stored in the validated folder while records that do not meet validation criteria or that are marked as duplicates are returned to the quarantine folder. This helps in enhancing data quality before processing it in the Silver layer and helps to ensure that only reliable data is used for downstream analytics and reporting.

Azure Synapse Spark Notebooks were used to transform data and implement business rules in the Silver layer. The datasets from the Bronze layer were validated to determine the Recoverable Assets (RA), Magna Carta (MC) dead stock, stockout risks and affected Sales Orders. There are some additional calculations that were generated to support business decision making, including material consumption analysis, inventory ageing, calculation of provision amounts and risk indicators.

The Gold layer was added to enable analytical reporting and dashboard development. Azure Synapse Analytics was used to transform the data from the Silver layer into an analytics-ready model. These final datasets were then linked to Power BI to create interactive dashboards that offer insights into inventory status, assets that can be recovered, dead stock, materials that are expired, risk of stockout and impacted customer orders.

Overall, the implemented architecture successfully integrates Azure Data Factory, Azure Data Lake Storage Gen2, Azure Synapse Analytics, and Power BI into a single data engineering solution. The architecture allows for automated data processing, enhances data quality and delivers consistent analytical results to support PPG's inventory risk management.

5.3 Coding of System's Main Functions

This section details the technical implementation and operational code for the primary data pipeline modules. The included scripts represent the exact configurations executed within the Microsoft Azure environment. The overall system architecture is organized into three development modules aligned along the data processing lifecycle.

5.3.1 Silver Layer Data Transformation and Business Logic on Recoverable Assets (RA) Metrics and the Magna Carta (MC) Inventory Compliance

This section covers how the Silver Layer actually works, focusing on how it automates the calculations for RA metrics and MC inventory classes. Sitting right between the raw data and the final dashboards, this part of the pipeline takes Bronze layer data and figures out inventory health and financial risk. To do this, the system looks at three main things: consumption over the last 12 months, how long stock has been sitting idle, and expiration dates. The cleaned, updated data is then saved into organized folders, creating a solid foundation for the final reports.

(a) Data Ingestion from Validated Storage

The Silver Layer pipeline starts by pulling data from the validated Bronze staging folder, skipping the raw landing area to keep the workflow secure and modular. Because the upstream ingestion modules have already organized the directory paths, the transformation engine gets

clean datasets without any structural weirdness, missing keys, or duplicate records. The execution script mounts these data lake paths and reads the materials master metadata, inventory snapshots, and historical production logs into separate Apache Spark DataFrames. This setup and the clean staging paths are shown below in Figure 5.3.1.1.

```
1 storage = "ppgsg"
2 bronze = f"abfss://{bronze}@{storage}.dfs.core.windows.net/validated"
3
4 √ materials = spark.read.option("header", "true") \
5                 .option("inferSchema", "true") \
6                 .csv(f"{bronze}/materials/")
7
8 √ inventory = spark.read.option("header", "true") \
9                 .option("inferSchema", "true") \
10                .csv(f"{bronze}/inventory_snapshot/")
11
12 √ production = spark.read.option("header", "true") \
13                 .option("inferSchema", "true") \
14                 .csv(f"{bronze}/production_orders/")
15
16 print("Materials:", materials.count(), "rows")
17 print("Inventory:", inventory.count(), "rows")
18 print("Production:", production.count(), "rows")
19 materials.printSchema()
```

Figure 5.3.1.1 : Silver Layer Source Ingestion Configuration

(b) Evaluation of Recoverable Assets (Question 2)

The pipeline checks for inventory optimization by comparing current stock levels against the last 12 months of production demand. By using an outer join to combine current inventory snapshots with this annual consumption data, the system easily spots surplus stock that goes beyond normal daily needs. If a material's current balance is higher than its total 12-month consumption, the system flags it as True in the `is_recoverable_asset` column. The script also calculates an `excess_quantity` metric by simply subtracting the annual demand from the current inventory level, and you can see how these columns are filled in the DataFrame previews in Figures 5.3.1.2 and 5.3.1.3.

```

1  from pyspark.sql import functions as F
2
3  # Fixed timeline anchor so your added data dates compute accurately
4  today = "2026-05-27"
5  # Filter production to last 12 months from our data timeline
6  prod_12m = production.filter(
7      F.to_date("production_date") >= F.add_months(F.to_date(F.lit(today)), -12)
8  )
9  # Sum consumption per material
10 consumption_12m = prod_12m.groupBy("material_id").agg(
11     F.sum("quantity_consumed").alias("consumption_12m")
12 )
13 # Get latest inventory snapshot per material
14 latest_inv = inventory.orderBy("snapshot_date", ascending=False) \
15     .dropDuplicates(["material_id"])
16
17 # Join and fill nulls (materials with no consumption = 0)
18 inv_cons = latest_inv.join(consumption_12m, "material_id", "left") \
19     .fillna(0, subset=["consumption_12m"])
20 # Apply exact blueprint RA flags
21 inv_ra = inv_cons.withColumn(
22     "is_recoverable_asset",
23     F.col("quantity_on_hand") > F.col("consumption_12m")
24 ).withColumn(
25     "excess_quantity",
26     F.col("quantity_on_hand") - F.col("consumption_12m")
27 )
28 print("=== Q2: Recoverable Assets ===")
29 inv_ra.filter("is_recoverable_asset = true") \
30     .select("material_id", "quantity_on_hand",
31             "consumption_12m", "excess_quantity").show()

```

Figure 5.3.1.2: Recoverable Assets (Q2) Pipeline Logic

```

=== Q2: Recoverable Assets ===
+-----+-----+-----+-----+
|material_id|quantity_on_hand|consumption_12m|excess_quantity|
+-----+-----+-----+-----+
| MAT026|3886|0|3886|
| MAT032|1132|0|1132|
| MAT084|572|0|572|
| MAT033|3064|0|3064|
| MAT099|525|0|525|
| MAT036|2404|0|2404|
| MAT039|2478|0|2478|
| MAT021|286|0|286|
| MAT065|21213|0|21213|
| MAT014|392|0|392|
| MAT066|20262|0|20262|
| MAT088|790|0|790|
| MAT059|23499|0|23499|
| MAT029|2676|0|2676|
| MAT074|17095|0|17095|
| MAT090|624|0|624|
| MAT049|3000|0|3000|
| MAT080|640|0|640|
| MAT083|633|0|633|
| MAT034|2391|0|2391|
+-----+-----+-----+-----+
only showing top 20 rows

```

Figure 5.3.1.3: Console Sample View of Q2

(c) Identification of Magna Carta Dead Stock (Question 3)

To track how long items have been sitting idle in the warehouse, the pipeline counts the months since each material's last transaction, using a default of 999 months for items that have

never been used. Following corporate rules, inventory is flagged as potential dead stock after nine months of zero activity. To prevent perishable, lot-controlled items from getting hit with incorrect financial write-downs, the system uses a quick two-part check: the `is_mc_dead_stock` flag only triggers if an item has been idle for over nine months and doesn't have an expiration date. You can see this filtering logic and the final data tables below in Figures 5.3.1.4 and 5.3.1.5.

```

1 # Last consumption date per material
2 last_consumed = production.groupBy("material_id").agg(
3     F.max("production_date").alias("last_consumed_date")
4 )
5 # Months since last consumption against our timeline date
6 last_consumed = last_consumed.withColumn(
7     "months_since_consumed",
8     F.months_between(F.to_date(F.lit(today)), F.to_date("last_consumed_date"))
9 )
10 # Materials never consumed get 999 months
11 never_consumed = materials.select("material_id") \
12     .join(last_consumed, "material_id", "left") \
13     .filter(F.col("last_consumed_date").isNull()) \
14     .withColumn("months_since_consumed", F.lit(999.0)) \
15     .withColumn("last_consumed_date", F.lit(None).cast("date"))
16
17 # Combine both streams
18 all_consumption = last_consumed.union(
19     never_consumed.select("material_id", "last_consumed_date", "months_since_consumed")
20 )
21 # Join with inventory RA result
22 inv_mc = inv_ra.join(all_consumption, "material_id", "left")
23 # MC Dead Stock rule from your blueprint
24 inv_mc = inv_mc.withColumn(
25     "is_mc_dead_stock",
26     (F.col("months_since_consumed") >= 9) &
27     (F.col("lot_expiry_date").isNull())
28 )
29 print("=== Q3: MC Dead Stock ===")
30 inv_mc.filter("is_mc_dead_stock = true") \
31     .select("material_id", "months_since_consumed",
32         "lot_expiry_date", "is_mc_dead_stock").show()

```

Figure 5.3.1.4: Compilation of Magna Carta Dead Stock Flags

```

=== Q3: MC Dead Stock ===
+-----+-----+-----+-----+
|material_id|months_since_consumed|lot_expiry_date|is_mc_dead_stock|
+-----+-----+-----+-----+
| MAT026|          999.0|         NULL|         true|
| MAT027|          999.0|         NULL|         true|
| MAT028|          999.0|         NULL|         true|
| MAT029|          999.0|         NULL|         true|
| MAT030|          999.0|         NULL|         true|
| MAT031|          999.0|         NULL|         true|
| MAT032|          999.0|         NULL|         true|
| MAT033|          999.0|         NULL|         true|
| MAT034|          999.0|         NULL|         true|
| MAT035|          999.0|         NULL|         true|
| MAT036|          999.0|         NULL|         true|
| MAT037|          999.0|         NULL|         true|
| MAT038|          999.0|         NULL|         true|
| MAT039|          999.0|         NULL|         true|
| MAT040|          999.0|         NULL|         true|
| MAT041|          999.0|         NULL|         true|
| MAT042|          999.0|         NULL|         true|
| MAT043|          999.0|         NULL|         true|
| MAT044|          999.0|         NULL|         true|
| MAT045|          999.0|         NULL|         true|
+-----+-----+-----+-----+
only showing top 20 rows

```

Figure 5.3.1.5: Console Sample View of Q3

(d) Expiry Auditing and Financial Provisions (Questions 4 & 5)

The pipeline tracks product freshness by comparing batch expiration dates against the current system date. Any materials that have already expired are flagged as `is_mc_expired` and pulled from active distribution. For items marked as either dead stock or expired, a 100% financial write-down is automatically applied. To do this, the transformation script pulls unit costs from the material master catalog and calculates the total liability column, `mc_provision_amount`, by multiplying the remaining on-hand stock by its unit cost. The underlying code blocks and the resulting audited write-down summaries are shown in Figures 5.3.1.6, 5.3.1.7, and 5.3.1.8.

```

3  inv_mc = inv_mc.withColumn(
4    "is_mc_expired",
5    (F.col("lot_expiry_date").isNotNull() & (F.to_date("lot_expiry_date") < F.to_date(F.lit(today))) |
6     (F.col("months_since_consumed") >= 12)
7  )
8  # Join with materials to get unit_cost
9  inv_full = inv_mc.join(
10   materials.select("material_id", "material_name", "category", "unit_cost"),
11   "material_id", "left"
12 )
13 # MC provision = unit_cost x quantity_on_hand (100% write-down)
14 inv_full = inv_full.withColumn(
15   "is_mc_material",
16   F.col("is_mc_dead_stock") | F.col("is_mc_expired")
17 ).withColumn(
18   "mc_provision_amount",
19   F.when(F.col("is_mc_material"),
20     F.col("quantity_on_hand") * F.col("unit_cost")
21   ).otherwise(0.0)
22 )
23 print("=== Q4: MC Expired Materials ===")
24 inv_full.filter("is_mc_expired = true") \
25   .select("material_id", "material_name",
26     "lot_expiry_date", "quantity_on_hand").show()
27 print("=== Q5: Total MC Provision Amount ===")
28 inv_full.agg(F.sum("mc_provision_amount") \
29   .alias("total_mc_provision_USD")).show()

```

Figure 5.3.1.6: Valuation and Processing of Expired Lots and Financial Write-Down Provisions

```

=== Q4: MC Expired Materials ===
+-----+
|material_id|      material_name|lot_expiry_date|quantity_on_hand|
+-----+
|  MAT026| White Spirit (Pure)|      NULL|      3886|
|  MAT032|   Steel Drum 1L|      NULL|      1132|
|  MAT084| Light Cyan Pigment| 2026-06-01|      572|
|  MAT033| Standard Cyan Paste|      NULL|     3064|
|  MAT099|   Vinyl Emulsion| 2026-06-01|      525|
|  MAT004| Phthalocyanine Blue| 2024-12-01|      435|
|  MAT036| Vibrant Red Pigment|      NULL|     2404|
|  MAT039|   Melamine Binder|      NULL|     2478|
|  MAT021|   Ethanol (Grade A)| 2024-12-01|      286|
|  MAT065|   Heptane (Grade A)| 2027-12-31|    21213|
|  MAT014|         Acetone| 2024-12-01|       392|
|  MAT066|   Ethanol (Standard)| 2027-12-31|    20262|
|  MAT088| Polyester Hardener| 2026-06-01|       790|
|  MAT059|   Ethanol (Standard)| 2027-12-31|   23499|
|  MAT012|         Toluene| 2024-12-01|       465|
|  MAT017|   Metal Can 4L| 2024-12-01|       423|
|  MAT029|   Metal Can Medium|      NULL|     2676|
|  MAT003|   Carbon Black| 2024-12-01|       179|
|  MAT074| Plastic Bottle 500ml| 2027-12-31|    17095|
|  MAT090|   Steel Drum 2L| 2026-06-01|       624|
+-----+
only showing top 20 rows

```

Figure 5.3.1.7: Console Sample View of Q4

```

=== Q5: Total MC Provision Amount ===
+-----+
|total_mc_provision_USD|
+-----+
|      4186539.750000001|
+-----+

```

Figure 5.3.1.8: Console Sample View of Q5

5.3.2 Silver Layer Data Transformation and Business Logic on Stockout Risks

This section details the implementation of the Silver Layer transformation to determine stockout risks and impacted sales. It answers which materials are below their reorder level, which materials are at risk of stockout based on consumption rate and which open sales orders and customers are affected materials that are potential stockout risk.

(a) Reorder Level Detection (Q1)

A fixed date anchor today = "2026-05-27" is used to ensure that all date-based calculations in this notebook match the generated data. It then extracts the most recent inventory snapshot per material by sorting by snapshot_date descending and removing duplicates by material_id. The result is then joined with the materials reference table to attach the reorder level

and lead time attributes. The `is_below_reorder` boolean rule is applied last by comparing `quantity_on_hand` to `reorder_level`. Materials that fail the reorder threshold are shown in the output cell. Figures 5.3.2.2 and 5.3.2.3 illustrate the code snippet and output respectively.

```
from pyspark.sql import functions as F

# Fixed timeline anchor – MUST match M3's value exactly
today = "2026-05-27"

# Get latest snapshot per material
latest_inv = inventory.orderBy("snapshot_date", ascending=False) \
    .dropDuplicates(["material_id"])

# Join with materials for reorder_level and lead_time_days
inv_mat = latest_inv.join(
    materials.select("material_id", "material_name", "category",
                    "reorder_level", "lead_time_days", "unit_cost"),
    "material_id", "left"
)

# Flag below reorder (M4 OWNS THIS COLUMN)
inv_mat = inv_mat.withColumn(
    "is_below_reorder",
    F.col("quantity_on_hand") < F.col("reorder_level")
)

print("=== Q1: Materials Below Reorder Level ===")
inv_mat.filter("is_below_reorder = true") \
    .select("material_id", "material_name", "category",
            "quantity_on_hand", "reorder_level").show()
```

Figure 5.3.2.2 Reorder Level Detection Code Snippet

```

=== Q1: Materials Below Reorder Level ===
+-----+-----+-----+-----+-----+
|material_id| material_name| category|quantity_on_hand|reorder_level|
+-----+-----+-----+-----+-----+
| MAT001| Titanium Dioxide| Pigment| 276| 500|
| MAT002| Iron Oxide Red| Pigment| 183| 300|
| MAT003| Carbon Black| Pigment| 179| 200|
| MAT006| Alkyd Resin| Resin| 381| 800|
| MAT007| Acrylic Resin| Resin| 347| 600|
| MAT008| Epoxy Resin| Resin| 104| 400|
| MAT011| Xylene| Solvent| 209| 1000|
| MAT012| Toluene| Solvent| 465| 800|
| MAT013| Mineral Spirits| Solvent| 210| 600|
| MAT014| Acetone| Solvent| 392| 400|
| MAT015| Butanol| Solvent| 211| 300|
| MAT016| Metal Can 1L|Packaging| 349| 2000|
| MAT017| Metal Can 4L|Packaging| 423| 1500|
| MAT018| Plastic Drum 20L|Packaging| 448| 500|
| MAT019| Cardboard Box|Packaging| 462| 1000|
| MAT020| Plastic Pail 5L|Packaging| 399| 800|
| MAT021|Ethanol (Grade A)| Solvent| 286| 1000|
| MAT022| Naphtha (Pure)| Solvent| 184| 1000|
| MAT023| Glass Bottle 25L|Packaging| 312| 800|
| MAT024|Cardboard Box 10L|Packaging| 287| 500|
+-----+-----+-----+-----+-----+
only showing top 20 rows

```

Figure 5.3.2.3 Reorder Level Detection Output

(b) Stockout Risk Calculation (Q6)

The code in Figure 5.3.2.4 derives stockout risk in three stages. First, it aggregates the production-orders table per material to obtain total consumption using first and last production dates, then divides total consumption by the number of active consumption days to produce an avg_daily_consumption rate column. The result is then joined back onto the inventory frame and fills nulls with a near-zero sentinel of 0.001 to prevent division-by-zero for materials with no production history and produces stock_cover_days as quantity-on-hand divided by daily consumption, with a maximum value of 9999.0 for materials tha has never been consumed. Finally, the is_stockout_risk flag is set when stock_cover_days falls below the supplier's lead_time_days where material will be consumed before a replenishment can arrive. The results of the code is shown in Figure 5.3.2.5.

```

# Total consumption and date range per material
cons_stats = production.groupBy("material_id").agg(
    F.sum("quantity_consumed").alias("total_consumed"),
    F.min("production_date").alias("first_prod_date"),
    F.max("production_date").alias("last_prod_date")
)

# Average daily consumption
cons_stats = cons_stats.withColumn(
    "days_active",
    F.datediff(F.to_date("last_prod_date"), F.to_date("first_prod_date")) + 1
).withColumn(
    "avg_daily_consumption",
    F.col("total_consumed") / F.col("days_active")
)

# Join with inventory + materials
stockout = inv_mat.join(cons_stats, "material_id", "left") \
    .fillna(0.001, subset=["avg_daily_consumption"])

# Days of stock remaining
stockout = stockout.withColumn(
    "stock_cover_days",
    F.when(F.col("avg_daily_consumption") > 0,
        F.col("quantity_on_hand") / F.col("avg_daily_consumption")
    ).otherwise(9999.0)
)

# Stockout risk: stock cover < lead time
stockout = stockout.withColumn(
    "is_stockout_risk",
    F.col("stock_cover_days") < F.col("lead_time_days")
)

print("=== Q6: Materials at Stockout Risk ===")
stockout.filter("is_stockout_risk = true") \
    .select("material_id", "material_name", "quantity_on_hand",
        "stock_cover_days", "lead_time_days").show()

```

Figure 5.3.2.4 Stockout Risk Calculation Code Snippet

```

=== Q6: Materials at Stockout Risk ===
+-----+-----+-----+-----+-----+
|material_id|    material_name|quantity_on_hand|  stock_cover_days|lead_time_days|
+-----+-----+-----+-----+-----+
|  MAT001|  Titanium Dioxide|          276|  4.828304668304669|          14|
|  MAT002|   Iron Oxide Red|          183|  4.0553072625698325|          10|
|  MAT003|   Carbon Black|          179|  3.9339420654911836|           7|
|  MAT004|Phthalocyanine Blue|          435| 11.670731707317072|          21|
|  MAT006|   Alkyd Resin|          381|  5.637075718015665|          14|
|  MAT007|   Acrylic Resin|          347|  7.6693630573248415|          14|
|  MAT008|   Epoxy Resin|          104|  1.7619417475728154|          21|
|  MAT011|       Xylene|          209|  4.00759355367386|           7|
|  MAT013|  Mineral Spirits|          210|  4.165760869565217|           7|
|  MAT015|       Butanol|          211|  7.124675324675325|          10|
|  MAT016|   Metal Can 1L|          349|  5.340063377093707|          14|
|  MAT017|   Metal Can 4L|          423|  8.87937576499388|          14|
|  MAT018| Plastic Drum 20L|          448|  9.964807930607186|          10|
+-----+-----+-----+-----+-----+

```

Figure 5.3.2.5 Stockout Risk Calculation Output

(c) Sales Impact Tracing (Q7)

This cell tracks supply risk from materials at risk with an open sales-orders layer by connecting a finished_product-to-material_id link table derived from production orders. Before the join is completed, both sides of each join key are normalized with trim and upper and a left-anti join is run first to identify any open sales orders with finished products that do not match any produced product. The cell then performs an inner join between open orders and at-risk materials to create the impacted-orders frame where the outputs are both the impacted orders and a distinct list of affected customers to support, as shown in Figures 5.3.2.6, 5.3.2.7, and 5.3.2.8.

```
# At-risk materials only
at_risk = stockout.filter("is_stockout_risk = true") \
    .select("material_id", "material_name", "stock_cover_days", "lead_time_days")

# Open sales orders only - normalise the join key
open_orders = sales.filter(F.upper(F.trim(F.col("status"))) == "OPEN") \
    .withColumn("fp_key", F.upper(F.trim(F.col("finished_product"))))

# Link finished_product -> material_id via production_orders (same normalisation)
prod_link = production.select(
    "material_id",
    F.upper(F.trim(F.col("finished_product"))).alias("fp_key")
).distinct()

# Show any sales products that DON'T match a production product (was silent before)
unmatched = open_orders.join(prod_link, "fp_key", "left_anti")
print("=== Unmatched finished_product values (check these!) ===")
unmatched.select("sales_order_id", "finished_product").show(truncate=False)

# Join open orders with material link
orders_mat = open_orders.join(prod_link, "fp_key", "left")

# Find orders needing at-risk materials
impacted = orders_mat.join(at_risk, "material_id", "inner")

print("=== Q7: Impacted Sales Orders ===")
impacted.select(
    "sales_order_id", "customer_name", "finished_product",
    "material_name", "required_delivery_date",
    "quantity_ordered", "stock_cover_days"
).show()

print("=== Affected Customers ===")
impacted.select("customer_name").distinct().show()
```

Figure 5.3.2.6 Sale Impact Tracing Code Snippet

```

=== Q7: Impacted Sales Orders ===
+-----+-----+-----+-----+-----+-----+
|sales_order_id|customer_name|finished_product|material_name|required_delivery_date|quantity_ordered|stock_cover_days|
+-----+-----+-----+-----+-----+-----+
|SO-003|IJM Corporation|Industrial Grey P...|Alkyd Resin|2025-01-13|381|5.637075718015665|
|SO-003|IJM Corporation|Industrial Grey P...|Butanol|2025-01-13|381|7.124675324675325|
|SO-003|IJM Corporation|Industrial Grey P...|Acrylic Resin|2025-01-13|381|7.6693630573248415|
|SO-003|IJM Corporation|Industrial Grey P...|Metal Can 1L|2025-01-13|381|5.340063377093707|
|SO-003|IJM Corporation|Industrial Grey P...|Plastic Drum 20L|2025-01-13|381|9.964807930607186|
|SO-003|IJM Corporation|Industrial Grey P...|Titanium Dioxide|2025-01-13|381|4.828304668304669|
|SO-003|IJM Corporation|Industrial Grey P...|Mineral Spirits|2025-01-13|381|4.165760869565217|
|SO-003|IJM Corporation|Industrial Grey P...|Epoxy Resin|2025-01-13|381|1.7619417475728154|
|SO-003|IJM Corporation|Industrial Grey P...|Metal Can 4L|2025-01-13|381|8.87937576499388|
|SO-003|IJM Corporation|Industrial Grey P...|Phthalocyanine Blue|2025-01-13|381|11.670731707317072|
|SO-003|IJM Corporation|Industrial Grey P...|Iron Oxide Red|2025-01-13|381|4.0553072625698325|
|SO-003|IJM Corporation|Industrial Grey P...|Xylene|2025-01-13|381|4.00759355367386|
|SO-003|IJM Corporation|Industrial Grey P...|Carbon Black|2025-01-13|381|3.9339420654911836|
|SO-007|Top Glove Facilities|Premium White Pai...|Epoxy Resin|2025-01-14|68|1.7619417475728154|
|SO-007|Top Glove Facilities|Premium White Pai...|Titanium Dioxide|2025-01-14|68|4.828304668304669|
|SO-007|Top Glove Facilities|Premium White Pai...|Mineral Spirits|2025-01-14|68|4.165760869565217|
|SO-007|Top Glove Facilities|Premium White Pai...|Xylene|2025-01-14|68|4.00759355367386|
|SO-007|Top Glove Facilities|Premium White Pai...|Phthalocyanine Blue|2025-01-14|68|11.670731707317072|
|SO-007|Top Glove Facilities|Premium White Pai...|Metal Can 4L|2025-01-14|68|8.87937576499388|
|SO-007|Top Glove Facilities|Premium White Pai...|Plastic Drum 20L|2025-01-14|68|9.964807930607186|
+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

```

Figure 5.3.2.7 List of Impacted Orders Output

```

=== Affected Customers ===
+-----+
|customer_name|
+-----+
|Sunway Building M...|
|Kerjaya Bina Bhd|
|SP Setia Hardware|
|Nippon Constructi...|
|Maybank Facilitie...|
|UEM Sunrise Bhd|
|BuildRight Hardware|
|Top Glove Facilities|
|Sime Darby Property|
|Gamuda Trading|
|YTL Hardware Centre|
|Metro Paint Supplies|
|IJM Corporation|
|Pavilion Retail S...|
|Eco World Supplies|
+-----+

```

Figure 5.3.2.8 List of Affected Customers Output

(d) Writing Stockout Outputs to Silver

This cell writes two Parquet datasets to the Silver container where `stockout_risk` contains the full inventory frame at material grain and is used by the merge cell that follows, while

impacted_sales_orders contains the customer-impact analysis at sales-order shown in Figure 5.3.2.9.

```
silver = f"abfss://silver@{storage}.dfs.core.windows.net"

# Stockout risk table (includes is_below_reorder, is_stockout_risk, etc.)
stockout.write.mode("overwrite").parquet(f"{silver}/stockout_risk/")

# Impacted sales orders table (M6 reads this directly for Q7)
impacted.write.mode("overwrite").parquet(f"{silver}/impacted_sales_orders/")

print("Silver stockout layer written.")
print(f"Stockout rows: {stockout.count()}")
print(f"Impacted order rows: {impacted.count()}")
```

Figure 5.3.2.9 Writing Output to Silver Code Snippet

5.3.3 Gold Layer Table

This module serves as the relational semantic layout for the entire reporting platform. Implemented entirely within an Azure Synapse Analytics Serverless SQL Pool, this layer converts the consolidated Parquet structures from the Silver container into a high performance analytics model. The pipeline sets up secure file formats and maps external source directories using credential networks. It extracts dimension tables directly from clean Bronze folders and maps foreign key relations to a centralized fact table (fact_inventory_status). Each code segment is detailed as in the next section.

5.3.3.1 Database Initialisation

Source code in *Figure 5.3.3.1.1* sets up a dedicated logical schema container namely as ppg_gold inside the Azure Synapse Analytics Serverless SQL environment. By using the USE statement, it will shift the execution context of the query engine to the database. This step will prevent namespace conflicts from happening as it ensures a clean permission pooling through Role-Based Access Control (RBAC) which also serves as a discrete connection target for the Power BI data gateway.

```
1  -- 1. Create dedicated analytics database
2  CREATE DATABASE ppg_gold;
3  GO
4
5  USE ppg_gold;
6  GO
```

Figure 5.3.3.1 : Database Initialisation Source Code

5.3.3.2 Dimensional Modeling Architecture

Figure 5.3.3.2 demonstrates the full implementation in the development of the dimensions table where this combined block represents the execution of the CETAS (Create External Table As Select) pattern to build the master dimension layers within the warehouse. It virtualizes and transforms flat source assets directly out of the clean Bronze lakehouse folders and writes them out into highly optimized physical target paths in the Gold container.

```

26 -----
27 -- 1. DIM_MATERIAL (Bronze -> Gold)
28 -----
29 IF OBJECT_ID('dim_material') IS NOT NULL DROP EXTERNAL TABLE dim_material;
30 GO
31
32 CREATE EXTERNAL TABLE dim_material
33 WITH ( LOCATION = 'dim_material', DATA_SOURCE = eds_tgt_gold_write, FILE_FORMAT = ff_csv_utf8 )
34 AS
35 SELECT
36     CAST(material_id AS VARCHAR(20)) COLLATE Latin1_General_100_BIN2_UTF8 AS material_id,
37     CAST(material_name AS VARCHAR(100)) COLLATE Latin1_General_100_BIN2_UTF8 AS material_name,
38     CAST(category AS VARCHAR(50)) COLLATE Latin1_General_100_BIN2_UTF8 AS category,
39     CAST(unit_of_measure AS VARCHAR(20)) COLLATE Latin1_General_100_BIN2_UTF8 AS unit_of_measure,
40     CAST(reorder_level AS INT) AS reorder_level,
41     CAST(lead_time_days AS INT) AS lead_time_days,
42     CAST(unit_cost AS FLOAT) AS unit_cost
43 FROM OPENROWSET(
44     BULK '/validated/materials/*.csv', DATA_SOURCE = 'eds_src_bronze', FORMAT = 'CSV', PARSER_VERSION = '2.0', FIRSTROW = 2
45 ) WITH (
46     material_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8, material_name VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
47     category VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8, unit_of_measure VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
48     reorder_level INT, lead_time_days INT, unit_cost FLOAT
49 ) AS r;
50 GO
51
52 -----
53 -- 2. DIM_SUPPLIER (Bronze -> Gold)
54 -----
55 IF OBJECT_ID('dim_supplier') IS NOT NULL DROP EXTERNAL TABLE dim_supplier;
56 GO
57
58 CREATE EXTERNAL TABLE dim_supplier
59 WITH ( LOCATION = 'dim_supplier', DATA_SOURCE = eds_tgt_gold_write, FILE_FORMAT = ff_csv_utf8 )
60 AS
61 SELECT
62     CAST(supplier_id AS VARCHAR(20)) COLLATE Latin1_General_100_BIN2_UTF8 AS supplier_id,
63     CAST(supplier_name AS VARCHAR(100)) COLLATE Latin1_General_100_BIN2_UTF8 AS supplier_name,
64     CAST(country AS VARCHAR(50)) COLLATE Latin1_General_100_BIN2_UTF8 AS country,
65     CAST(reliability_rating AS VARCHAR(10)) COLLATE Latin1_General_100_BIN2_UTF8 AS reliability_rating
66 FROM OPENROWSET(
67     BULK '/validated/suppliers/*.csv', DATA_SOURCE = 'eds_src_bronze', FORMAT = 'CSV', PARSER_VERSION = '2.0', FIRSTROW = 2
68 ) WITH (
69     supplier_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8, supplier_name VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
70     country VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8, reliability_rating VARCHAR(10) COLLATE Latin1_General_100_BIN2_UTF8
71 ) AS r;
72 GO
73
74 -----
75 -- 3. DIM_CUSTOMER (Bronze -> Gold) - Expanded Data
76 -----
77 IF OBJECT_ID('dim_customer') IS NOT NULL DROP EXTERNAL TABLE dim_customer;
78 GO
79
80 CREATE EXTERNAL TABLE dim_customer
81 WITH (
82     LOCATION = 'dim_customer', -- Created cleanly inside Gold container
83     DATA_SOURCE = eds_tgt_gold_write, -- WRITES TO GOLD
84     FILE_FORMAT = ff_csv_utf8
85 )
86 AS
87 SELECT DISTINCT
88     CAST(customer_name AS VARCHAR(100)) COLLATE Latin1_General_100_BIN2_UTF8 AS customer_name
89 FROM OPENROWSET(
90     BULK '/validated/sales_orders/*.csv', -- Pulls from updated 1,000 row source files
91     DATA_SOURCE = 'eds_src_bronze',
92     FORMAT = 'CSV',
93     PARSER_VERSION = '2.0',
94     FIRSTROW = 2
95 ) WITH (
96     customer_name VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8
97 ) AS r;
98 GO

```

Figure 5.3.3.2: Dimensional Modeling Architecture (*dim_material*, *dim_supplier*, and *dim_customer*)

Every text and numeric attribute passes through an explicit type transformation using CAST. The raw fields are hardened into strict architectural objects by using VARCHAR, INT and FLOAT. This establishes strict memory boundaries and data type consistency across all dimension blocks.

In Azure Synapse Analytics, the distributed joins between un-collated CSV metadata strings and structured Parquet formats will trigger a distributed query processing (DQP) failure or data mismatch error. By enforcing explicit UTF-8 binary collation (COLLATE Latin1_General_100_BIN2_UTF8) it will standardize the case sensitivity across data silos, ensuring low-latency joining behaviors.

In dim_customer source code, SELECT DISTINCT is implemented as a calculation loop on the raw sales order file streams. This approach is implemented as the original transactional logs are repetitive across lines, this step filters out the noise to build a clean customer dimension table. This satisfies the 1,000-row expanded synthetic use case parameters and provides clear, cross-functional visibility for customer order runout tracing.

5.3.3.3 Dynamic Time Dimension Virtualization

Unlike other dimension tables, dim_date does not implement CETAS method as it generates a virtualized calendar table that does not require physical storage. It constructs an abstract numeric sequence via a succession of recursive binary Cartesian cross-joins (CROSS JOIN paths t1 through t5), building a series of over 65,000 incremental values. The DayOffset then seeds a DATEADD function to generate an unbroken date timeline from January 1, 2025, through December 31, 2026. This raw axis is parsed into operational attributes using DATEPART commands (month, quarter, year). This approach will ensure that there are no gaps in time-series logic. Besides, this table acts as the backbone for all inventory aging dashboards and supports the rolling calculations required during quarterly Recoverable Asset validations.

```

100 -----
101 -- 4. DIM_DATE (Dynamic Structural View)
102 -----
103 CREATE OR ALTER VIEW dim_date AS
104 WITH
105 t1 AS (SELECT 1 AS n UNION ALL SELECT 1),           -- 2 rows
106 t2 AS (SELECT 1 AS n FROM t1 a CROSS JOIN t1 b),    -- 4 rows
107 t3 AS (SELECT 1 AS n FROM t2 a CROSS JOIN t2 b),    -- 16 rows
108 t4 AS (SELECT 1 AS n FROM t3 a CROSS JOIN t3 b),    -- 256 rows
109 t5 AS (SELECT 1 AS n FROM t4 a CROSS JOIN t4 b),    -- 65,536 rows
110 Numbers AS (
111     SELECT ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) - 1 AS DayOffset
112     FROM t5
113 ),
114 DateSequence AS (
115     SELECT DATEADD(day, DayOffset, CAST('2025-01-01' AS DATE)) AS date_id
116     FROM Numbers
117     WHERE DayOffset <= DATEDIFF(day, '2025-01-01', '2026-12-31')
118 )
119 SELECT
120     date_id,
121     DATEPART(month, date_id) AS [month],
122     DATEPART(quarter, date_id) AS [quarter],
123     DATEPART(year, date_id) AS [year]
124 FROM DateSequence;
125 GO

```

Figure 5.3.3.3 : dim_date Source Code

5.3.3.4 Central Fact Table Ingestion

This core statement generates the centralized measurement hub (fact_inventory_status) by ingesting the single unified Parquet dataset from the Silver tier directory path (/combined_inventory/). The source code also enforces a conditional case loops to govern inventory classification states and resolve flag conflicts:

- is_mc_expired checks if the physical lot shelf life has been breached ($lot_expiry_date < active_system_date$), evaluating to an absolute boolean true flag.
- is_mc_dead_stock evaluates if the item matches stagnation conditions ($months_since_consumed \geq 9$) only if the lot expiry date remains in the future ($lot_expiry_date > active_system_date$).

The conditional in the source code will guarantee the data governance correctness as it prevents an item from being simultaneously flagged as both active Dead Stock and Expired, ensuring absolute mathematical accuracy for downstream financial provisions and write-downs.

```

26 CREATE EXTERNAL TABLE fact_inventory_status
27 WITH (
28     LOCATION = 'fact_inventory_status',
29     DATA_SOURCE = eds_tgt_gold_write,      -- Writes straight to your Gold Container
30     FILE_FORMAT = ff_parquet_gold
31 )
32 AS
33 SELECT
34     -- Keys & Dates (Using M1's new attribute name)
35     CAST(r.material_id AS VARCHAR(20)) COLLATE Latin1_General_100_BIN2_UTF8 AS material_id,
36     CAST(r.snapshot_date AS DATE) AS snapshot_date,
37     CAST(r.lot_expiry_date AS DATE) AS lot_expiry_date,
38     CAST(r.warehouse_location AS VARCHAR(50)) COLLATE Latin1_General_100_BIN2_UTF8 AS warehouse_location,
39
40     -- Quantities & Baseline Metrics
41     CAST(r.quantity_on_hand AS INT) AS quantity_on_hand,
42     CAST(r.consumption_12m AS FLOAT) AS consumption_12m,
43     CAST(r.excess_quantity AS FLOAT) AS excess_quantity,
44     CAST(r.months_since_consumed AS INT) AS months_since_consumed,
45
46     -- M3 Flags (Recoverable Assets & Core Material)
47     CAST(r.is_below_reorder AS BIT) AS is_below_reorder,
48     CAST(r.is_recoverable_asset AS BIT) AS is_recoverable_asset,
49     CAST(r.is_mc_material AS BIT) AS is_mc_material,
50     CAST(r.mc_provision_amount AS FLOAT) AS mc_provision_amount,
51
52     -- ENFORCING PRIORITIZED CONDITIONAL RULES (Resolved Flag Conflict)
53     CAST(
54         CASE
55             WHEN r.lot_expiry_date < CAST(GETDATE() AS DATE) THEN 1
56             ELSE 0
57         END AS BIT
58     ) AS is_mc_expired, -- EXPIRED TAKES ABSOLUTE PRIORITY
59
60     CAST(
61         CASE
62             WHEN r.months_since_consumed >= 9 AND r.lot_expiry_date >= CAST(GETDATE() AS DATE) THEN 1
63             ELSE 0
64         END AS BIT
65     ) AS is_mc_dead_stock, -- ONLY DEAD STOCK IF NOT YET EXPIRED
66
67     -- M4 Flags (Stockout Risks Linked on Keys)
68     CAST(r.is_stockout_risk AS BIT) AS is_stockout_risk,
69     CAST(r.stock_cover_days AS FLOAT) AS stock_cover_days,
70     CAST(r.avg_daily_consumption AS INT) AS avg_daily_consumption
71 FROM OPENROWSET(
72     -- FIXED SOURCE PATH: Now pulling from Member 4's merged and unified silver output folder
73     BULK '/combined_inventory/**',
74     DATA_SOURCE = 'eds_ppg_silver',
75     FORMAT = 'PARQUET'
76 ) AS r;
77

```

Figure 5.3.3.4 : Database Initialisation Source Code

5.4 Essential Interfaces (The Dashboards & Tables)

This dashboard section is the last presentation layer in the proposed data engineering architecture. The data ingestion, transformation and storage processes ensure the inventory

information is accurately prepared, while the dashboard and reporting interfaces are used to interpret the transformed data through meaningful visualizations and operational reports. To support efficient analytics and decision-making, a semantic layer was first established using Azure Synapse Serverless SQL views before the data was presented through interactive Power BI dashboards.

5.4.1 Serverless SQL Views (Semantic Layer Design)

To separate concerns and have a well defined semantic layer between data engineering and data visualization, a few views (vw_q1 to vw_q7) were deployed as serverless SQL that map to the different data sources. All complex filtering, categorical mapping and financial calculation limits were moved from the loaded tables to the Power BI Synapse Serverless SQL pool. This design approach guarantees data consistency and maintains a light front-end presentation tier.

5.4.1.1 Semantic View Mapping and Logic Design Patterns

Each view was designed as a pre-computed data source tailored to answer the specific business requirements of the dashboard:

I. `dbo.vw_q1_below_reorder` (Operational Alerts):

This view does an inventory threshold analysis, which means it looks at materials and finds those that have `quantity_on_hand` less than a value defined by a `reorder_level`. The view filters records according to the inventory risk conditions to deliver a targeted set of records that are materials that need replenishment attention. The output data can then be used to inform operational monitoring and will feed into the Critical Storage Count KPI on the dashboard and inventory risk visualizations.

II. `dbo.vw_q2_mc_provisions` (Inventory Provision Analysis View):

This perspective is the main financial information source for inventory provision reporting. It uses business rules with conditional logic to categorize inventory records as Expire, Dead Stock and healthy. Moreover, the provision levels are determined by

organisational policies. Stock which is classified as Healthy is given a provision amount of RM0, whereas provision amounts of MC Expired and MC Dead Stock are limited to the inventory value. The view provides the financial exposure metrics that are shown on the dashboard page of the Obsolescence Analysis view.

III. `dbo.vw_q3_mc_dead_stock` (Dead Stock Audit View)

This view will capture the inventory records that meet the organization's dead stock rules, such as inventory that has not been consumed in any sales for 9 months or more. The view is used to act as an intermediate audit layer to validate inventory ageing and classification processes prior to aggregation into higher level reports.

IV. `dbo.vw_q4_mc-expired` (Expired Inventory Monitoring View):

This view will help find the inventory records that have expired. It gives a separate data set to track the stock that has expired and the loss, as well as to manage operations. For instance, inventory view, disposal planning and compliance monitoring. The view is not directly related to dashboard visualizations, but it helps warehouse operations.

V. `dbo.vw_q5_recoverable_assets` (Recoverable Assets View):

The view represents a semantic endpoint that filters and aggregates a specific set of stable, viable warehouse materials whose inventory consists of a safe level in relation to business demand. It extracts pre-calculated asset flags (`is_recoverable_asset=1`) from the underlying curated Silver layer, which clearly distinguishes live and active inventory records from dead or expired stock. This structured output creates a single source of truth for tracking high value recoverable assets, separate from the financial liabilities and is a clean, business-ready data stream for downstream dashboard metrics.

VI. `dbo.vw_q7_impacted_orders` (Supply Chain Risk Assessment View):

This perspective combined the inventory shortage data with the sales order data of the customers to determine orders that might be impacted by not having enough stock. The

view connects internal inventory risks to external customer commitments by using Azure Synapse Serverless SQL capabilities, such as being able to access sales order data in Azure Data Lake Storage directly. The resulting data can be used as input in the analysis of risks in the supply chain as it appears on the Supply Chain Stockout Alert dashboard.

5.4.1.2 Semantic View Mapping and Logic Design Patterns

Implementing business logic within Azure Synapse Serverless SQL views provided several advantages for the Power BI reporting environment.

I. Reduced Front-End Processing

Data transformations, including inventory classification, provision calculation and filtering, were handled in the Synapse SQL views rather than in Power BI. This will minimize the amount of Data Analysis Expressions (DAX) and Power Query transformations that are complex and will enable Power BI to be mainly a data visualization tool.

II. Optimized Data Loading

By leveraging targeted semantic views, only relevant and business-ready datasets were imported into Power BI. The dashboard ran on smaller data sets since it was not directly connected to large transactional tables, which improved refresh performance and made it easier to maintain reports.

III. Improved Data Consistency

Business rules were centralized in the SQL views such that the calculations and classifications were applied consistently across all the dashboard pages. This reduced the likelihood of discrepancies between visualizations and increased the reliability of the analytical results.

5.4.2 Power BI Dashboard Execution

This section represents the interactive Power BI dashboard that was developed as the final output of the data engineering pipeline. The dashboard serves as the primary decision-support tool for PPG's inventory risk management by translating the processed data from the Gold layer into a clear and actionable dashboard. It is structured into three pages, designed to answer a specific set of business questions related to inventory risk, material obsolescence and supply chain stockout impact.

The Power BI layer is built purely for a presentation basis. All metric calculations are pre-computed within the Synapse SQL views (vw_q1 to vw_q7) managed by Member 6. The SQL views created before designing the dashboard could make sure the dashboard created are consistent with the pipeline outputs and to prevent duplicate calculation logic exists within Power BI itself. Microsoft Power BI Desktop was connected to the Azure Synapse Analytics serverless SQL endpoint using Import mode with the database name ppg_gold as the data source.

5.4.2.1 Dashboard Page 1: Inventory Risk Overview

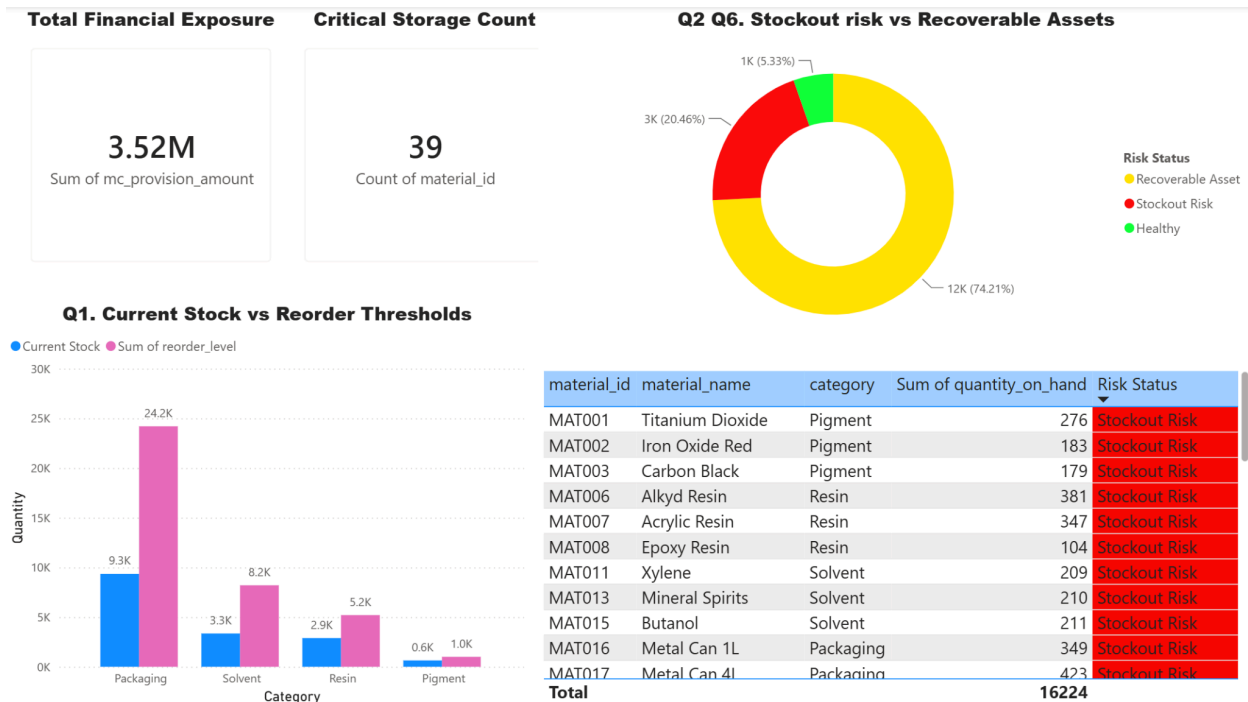


Figure 5.4.2.1 Inventory Risk Overview Dashboard

Figure 5.4.2.1 shows the first page of the Power BI Dashboard. It provides a detailed summary of the overall inventory risk status across all material categories. It is designed to give management a quick snapshot of the financial exposure and the distribution of stock health of each material. The page contains four key visual components.

At the top-left of the page, 2 KPI cards are set. The first card labelled “Total Financial Exposure” shows a total sum of RM3.52 million which is obtained from the vw_q2_mc_provision table's field mc_provision_amount. This is the total amount that is at risk from materials covered by the rules of Magna Carta (MC). The second KPI card “Critical Storage Count” displays the number of materials that are currently below reorder level. The value comes from the vw_q1_below_reorder which returned 39 materials.

In the lower-left corner of the page is a clustered bar chart titled “Current Stock vs Reorder Thresholds”. It displays the current quantity on hand (current stock) of the materials in comparison to the reorder level's materials for four material groups, namely Packaging, Solvent, Resin and Pigment.

It is clearly evident from the chart that the current stock of Packaging is high which is about 9300 and the lowest stock is of Pigment which is about 600. This visual allows the procurement team to easily see which categories are more likely to be at a critical stock level and in need of purchase.

In the upper right corner of the page, there is a donut chart showing the sum of quantity on hand by risk status. It divides all materials based on their designated risk factor, which are red (Stockout Risk), yellow(Recoverable Asset) and green (Healthy). As displayed in the chart, of the total quantity on hand, 74.21% falls under the Recoverable Asset category. This indicates that there is a significant surplus in the inventory. The percent of total current stock that are Stockout Risk is 20.46%. This shows the extent of the problem of inventory risk, as only 5.33% is in the Healthy category.

At the bottom right corner of the page there is a material detail table. It displays every material with the material ID, name, category, quantity and risk status. This data comes from vw_q1_below_reorder and fact_inventory_status. The Conditional Background Formatting has been applied to the Risk Status column, enabling the Stockout Risk rows to be highlighted in red and enabling analysts to see at a glance which materials are at risk of being out of stock and need immediate attention without the need for manually scanning each row.

5.4.2.2 Dashboard Page 2: MC Obsolescence Analysis

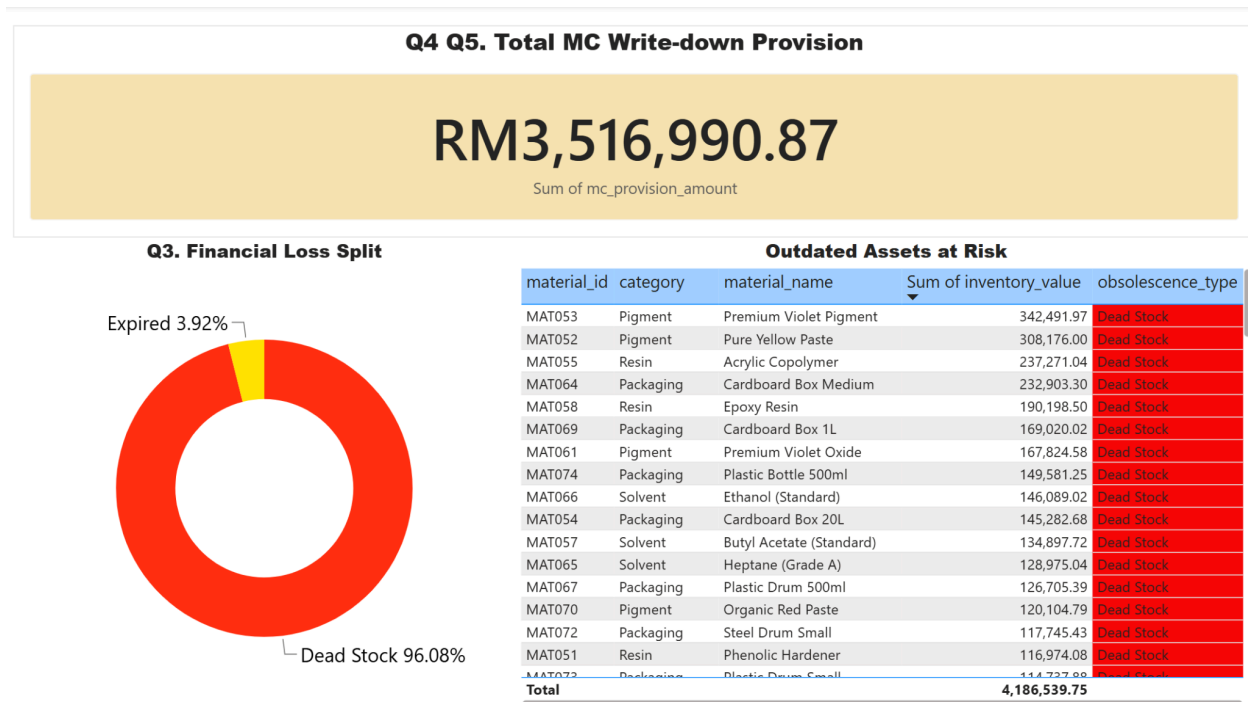


Figure 5.4.2.2 MC Obsolescence Dashboard

The second page of the operational Power BI dashboard is shown in Figure 5.4.2.2. It highlights materials under the Magna Carta (MC) write down rules. This dashboard filters for materials that meet the MC Dead Stock or MC Expired criteria to give the Finance Department a transparent and clear view of the liabilities and tracking of materials in the organisation under both risk categories.

At the top of the interface, there is a large KPI card that displays a general overview of the financial exposures. It reflects the total write down expenses needed to cover the current subset of inventory items that are at risk with a total RM3,516,990.87. This measure is a critical financial measure for the corporate finance team to determine the amount of reserves to set aside to cover material depreciation losses.

A donut chart entitled “Financial Loss Split” is placed on the left side of the canvas to show the exact composition of these liabilities. The visualization shows 96.08% of MC Dead Stock while MC Expired inventory accounts for the remaining 3.92% only. This summarization shows that the primary financial risk is not because of the materials that have passed their expiry date but are from

materials that have not been consumed in the past 9 months. The colours are assigned to the following risk colours: red for MC Dead Stock, yellow for MC Expired and green for Healthy assets which are aligned with the risk severity.

The right side of the dashboard is a detailed table of “Outdated Assets at Risk”. This table serves as a mapping of the material identifiers and information extracted from the material ID, name, category, inventory value and obsolescence type. This table is sorted by the most expensive obsolete materials at the top of the table. Premium Violet Pigment (MAT053) and Pure Yellow Paste (MAT052) are the top two material value items with RM342,491.97 and RM308,176.00 respectively. The column obsolescence_type has also been conditionally formatted to highlight MC Dead Stock in red, MC Expired in yellow and Healthy in green, so it's easy for the finance team to see the worst write-down categories at a glance. Finally, the bottom line of the table sums up to RM4,186,539.75, which is the total inventory costs (including healthy assets).

5.4.2.3 Dashboard Page 3: Supply Chain Stockout Alert

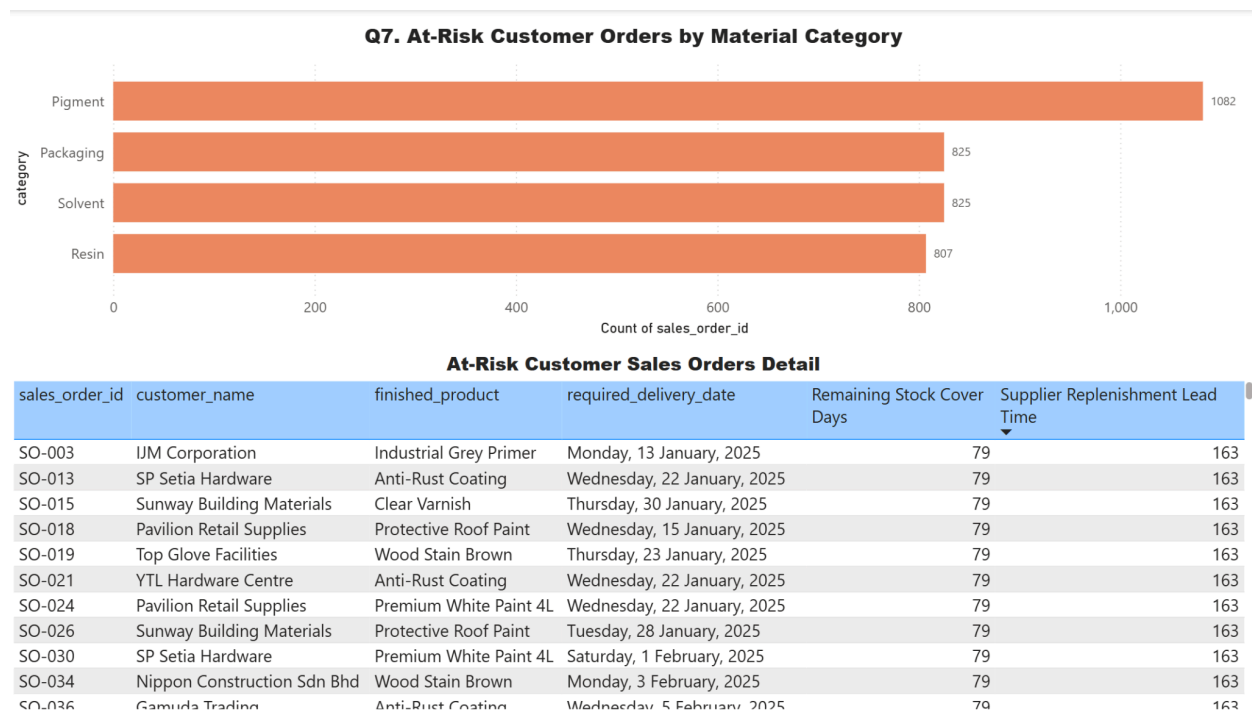


Figure 5.4.2.3 Supply Chain Stockout Alert Dashboard

Figure 5.4.2.3 shows the last page of the dashboard. It highlights the effect of material shortage on customer sales order. It is created for the Sales Department for comparing the stock levels of raw materials and alerting them of which customer orders are likely to be delayed.

The top half of the page shows a horizontal bar chart labeled “At-Risk Customer Orders by Material Category”. It is a graph that displays the number of at-risk sales order items in vw_q2_mc_provision and vw_q7_impacted_orders tables, broken down by material category. The chart shows that Pigment is the most affected category with 1,082 impacted orders, followed by Packaging and Solvent both at 825 orders and lastly Resin at 807 orders. A consistent orange colour is used for all bars, to reinforce the risk theme of the page. The chart is sorted by descending order, so that the replenishment actions of the supply chain team can be prioritised by category.

Lastly, the lower half of the page contains a detailed table, labeled “At-Risk Customer Sales Orders Detail”. It shows details of sales orders along with the Order Id, Customer Name, Finished

Product, Remaining Stock Cover Days and Supplier Replenishment Lead Time (days). All of this data is taken from vw_q7_impacted_orders. This proactive approach helps the sales team to handle customer expectations before specific delivery problems happen.

5.5 Testing (Black box, White box, and User Acceptance)

Black box testing treats a component as a sealed unit and verifies only that the inputs produce the expected outputs. In this pipeline it applies to data movement and loading stages bronze ingestion, the data-quality split, and the gold load.

White box testing inspects the internal logic of the transformation code and verifies that every branch behaves correctly. It applies to the PySpark notebooks that compute business flags and to the SQL views that pre-filter the fact table.

User Acceptance Testing (UAT) verifies that the final dashboard answers PPG's seven business questions correctly and is usable for the intended audience.

Table 5.1: Testing Roles and Responsibilities

Member	Pipeline Layer	Component Owned	Testing Type Assigned
M1	Bronze (ingestion)	ADF Copy pipeline	Black Box
M2	Bronze (validation)	Data Flow	Black Box
M3	Silver (RA/MC logic)	PySpark notebook	White Box
M4	Silver (stockout + merge)	PySpark notebook + merge cell	White Box
M5	Gold (modeling + loading)	Synapse SQL — tables + fact load	Black Box
M6	Gold (views)	Synapse SQL — 7 views	White Box
M7	Presentation	Power BI dashboard	UAT (whole team participates)

5.5.2 Black Box Testing

Black box tests verify components by comparing observed outputs against expected outputs without inspecting the implementation.

Bronze Ingestion

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass/Fail
BB-M1-01	All 6 source CSVs are copied to the ingested folder.	Trigger pl_bronze_ingestion. Wait for completion in the Monitor tab. List contents of bronze/ingested/.	All 6 subfolders (materials, inventory_snapshots, purchase_orders, production_orders, suppliers, sales_orders) exist with one CSV each.	Monitor tab confirms all 6 Copy Data activities completed with a 'Succeeded' status. Files verified in the target container.	Pass
BB-M1-02	Row counts after copy match source row counts.	For each file, compare row count in bronze/raw/ vs bronze/ingested/ using SELECT COUNT(*) on OPENROWSET.	Source row count = destination row count for all 6 files.	Queried row counts match exactly across all 6 datasets (0 rows lost during ingestion).	Pass

BB-M1-03	Pipeline fails gracefully when a source file is missing.	Temporarily delete materials.csv from bronze/raw/materials/. Re-trigger the pipeline.	Copy_Materials activity fails with a clear 'file not found' error; other 5 activities still succeed (no silent skip).	Pipeline registered a localized failure for the missing file. The remaining 5 files were successfully copied (no silent skip).	Pass
BB-M1-04	Linked services are healthy.	ADF → Manage Linked services → Click Test Connection on ls_adls_ppg and ls_synapse_ppg.	Both show 'Connection successful'.	Both linked services returned a 'Connection successful' status prompt.	Pass
BB-M1-05	Pipeline runs in acceptable time for the 6 files.	Record total duration shown in Monitor tab.	Total pipeline runtime is under 5 minutes for the sample dataset.	The 6 Copy Data activities completed in a maximum of 1 minute and 5 seconds, well below the 5-minute threshold.	Pass

BB-M1-01	All 6 source CSVs are copied to the ingested folder.	Trigger pl_bronze_ingestion. Wait for completion in Monitor tab. List contents of bronze/ingested/.	All 6 subfolders (materials, inventory_snapshot, purchase_orders, production_orders, suppliers, sales_orders) exist with one CSV each.	Monitor tab confirms all 6 Copy Data activities completed with a 'Succeeded' status. Files verified in the target container.	Pass
----------	--	--	--	--	------

Data Quality & Master Pipeline

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass/Fail
BB-M2-01	Rows with NULL material_id are quarantined.	Inject 3 rows with NULL material_id into materials test file. Run df_bronze_quality.	Exactly 3 rows appear in bronze/quarantine/materials/. Validated folder count = original – 3.	NULL material_id rows were quarantined successfully.	Pass
BB-M2-02	Rows with NULL unit_cost are quarantined.	Inject 2 rows with NULL unit_cost into materials test file. Run df_bronze_quality.	Exactly 2 rows appear in quarantine; the rest in validated.	NULL unit_cost rows were quarantined successfully.	Pass
BB-M2-03	Duplicate rows are detected and quarantined.	Inject 2 exact duplicate rows in purchase_orders (same po_id). Run the pipeline.	Duplicate rows land in bronze/quarantine/duplicates/. Validated folder contains only one of the pair.	Duplicate records were detected and quarantined.	Pass

BB-M2-04	Clean input produces empty quarantine.	Run df_bronze_quality on a verified clean file.	Quarantine folder is empty (file exists or no file, depending on implementation). Validated row count = source row count.	No records were quarantined; all records were validated.	Pass
BB-M2-05	Invalid data type values are quarantined.	Insert rows with invalid numeric values (e.g., text in quantity_on_hand). Run df_bronze_quality.	Invalid rows are routed to quarantine while valid rows remain in validated.	Invalid data type records were quarantined successfully.	Pass
BB-M2-06	Multiple validation rules work simultaneously.	Inject NULL values and duplicate records into the same dataset. Run df_bronze_quality.	Invalid and duplicate rows are quarantined. Valid records are written to validated.	Multiple invalid records were quarantined successfully.	Pass
BB-M2-07	End-to-end Data Quality pipeline execution succeeds.	Execute the Bronze Data Quality pipeline and monitor the output folders.	Pipeline status shows Succeeded. Validated and quarantine outputs are generated correctly.	Pipeline executed successfully with expected outputs.	Pass

Gold Layer Loading

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass/Fail
BB-M5-01	All four dimension tables exist in ppg_gold.	SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_CATALOG = 'ppg_gold';	Returns dim_material, dim_supplier, dim_customer, dim_date, fact_inventory_statuses.	Returns dim_material, dim_supplier, dim_customer, fact_inventory_statuses, vw_q1_below_reorder, vw_q2_recoverable_assets, vw_q3_mc_dead_stock, vw_q4_mc_expired, vw_q5_provision, vw_q7_impacted_orders, vw_q2_mc_provisions, dim_date	Pass
BB-M5-02	dim_material is populated with correct row count.	SELECT COUNT(*) FROM dim_material; Compare the row count of	Counts match exactly.	1000 rows is returns	Pass

		bronze/validate d/materials/.			
BB-M5-03	dim_supplier and dim_customer are populated.	SELECT COUNT(*) FROM dim_supplier; SELECT COUNT(*) FROM dim_customer;	Both return values > 0 and match the source distinct counts.	Return 1000 rows and match the distinct counts	Pass
BB-M5-04	dim_date is populated (Fix #3 verification).	SELECT COUNT(*) FROM dim_date; SELECT MIN(date_id), MAX(date_id) FROM dim_date;	Row count > 0. Min/Max dates match the snapshot_date range in combined_inventory.	The dates match the snapshot_date range in combined_inventory	Pass
BB-M5-05	fact_inventory_status has no NULL stockout columns (Fix #1 verification).	SELECT COUNT(*) FROM fact_inventory_status WHERE is_stockout_risk IS NULL; SELECT COUNT(*) FROM	Both queries return 0 — proving M4's merged columns loaded correctly.	Both queries return 0, proven M4's merged columns loaded correctly.	Pass

		fact_inventory_ status WHERE is_below_reord er IS NULL;			
BB- M5-0 6	fact_inventory_ status row count matches M4's combined output.	Run M4's print(combined. count()) in the notebook; record value X. SELECT COUNT(*) FROM fact_inventory_ status;	Both numbers are equal.	Both numbers are equal	Pass
BB- M5-0 7	Fact table loads from combined_inve ntory, not ra_mc_flags.	Inspect M5's INSERT INTO fact_inventory_ status statement.	OPENROWSET path is 'https://ppgsg.dfs.cor e.windows.net/silver /combined_inventor y/**' (NOT ra_mc_flags).	Same as the expected result	Pass

5.5.3 White Box Testing

White box tests inspect the internal logic of the code and verify every branch behaves as designed. Each branch in the business-logic code receives at least one targeted test case.

RA / MC Notebook Logic

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass/Fail
WB-M 3-01	RA flag - TRUE branch (excess inventory).	Insert test row: material_id='T01', quantity_on_hand=100, consumption_12m=60. Run Cell 2 logic.	is_recoverable_asset = true, excess_quantity = 40.	Row filtered successfully. Spark output matches expected metrics exactly	Pass
WB-M 3-02	RA flag - FALSE branch (consumption exceeds stock).	Insert test row: material_id='T02', quantity_on_hand=50, consumption_12m=80.	is_recoverable_asset = false, excess_quantity = -30.	Row filtered successfully. Spark output matches expected metrics exactly	Pass
WB-M 3-03	MC Dead Stock - TRUE branch.	Insert: material_id='T03', last consumption 10 months ago, lot_expiry_date = NULL.	months_since_consumed $\approx$ 10, is_mc_dead_stock = true.	Row filtered successfully. Spark script computes temporal gap metrics as	Pass

WB-M 3-04	MC Dead Stock - FALSE branch (recently consumed).	Insert: material_id='T04', last consumption 2 months ago, lot_expiry_date = NULL.	is_mc_dead_stock = false (months_since_consumed < 9).	Row filtered successfully. Temporal boundaries evaluated below corporate limit threshold	Pass
WB-M 3-05	MC Dead Stock - FALSE branch (has lot expiry).	Insert: material_id='T05', last consumption 15 months ago, lot_expiry_date = '2030-01-01'.	is_mc_dead_stock = false (lot_expiry_date IS NOT NULL excludes it).	Row filtered successfully. Non-null expiration date successfully overrides dead stock rules	Pass
WB-M 3-06	Never-consumed branch (sentinel value).	Insert material in materials but no rows in production_orders.	months_since_consumed = 999.0, is_mc_dead_stock evaluated with that value.	Unconsumed item isolated. Spark fillna() conditional rule maps fallback bounds precisely to: months_since_consumed = 999.0 is_mc_dead_stock = true	Pass
WB-M 3-07	MC Expired - TRUE branch.	Insert: material_id='T06', lot_expiry_date	is_mc_expired = true.	Row filtered successfully. System baseline comparison	Pass

		= '2024-01-01' (past date).		triggers stale lot validation rules	
WB-M 3-08	MC Expired - FALSE branch (future date).	Insert: material_id='T0 7', lot_expiry_date = '2030-12-31'.	is_mc_expired = false.	Row filtered successfully. Active catalog asset verified within safe boundaries	Pass
WB-M 3-09	MC provision = unit_cost × qty when is_mc_material is TRUE.	Insert: material_id='T0 8', is_mc_dead_st ock=true, quantity_on_ha nd=10, unit_cost=50.	mc_provision_amou nt = 500.0.	Conditional loop identifies asset risk. 100% financial write-down evaluates precisely to	Pass
WB-M 3-10	MC provision = 0 when is_mc_material is FALSE.	Insert: material_id='T0 9', both MC flags = false, quantity_on_ha nd=10, unit_cost=50.	mc_provision_amou nt = 0.0.	Row filtered successfully. Asset bypassed from risk write-down calculations	Pass
WB-M 3-11	Silver output written to correct path.	After Cell 5, list silver/ra_mc_fl ags/ in the portal.	Folder exists and contains .parquet files. Notebook printed row count matches expected.	Path verified via Storage Explorer framework. Folder contains all schema binaries and a	Pass

				successful _SUCCESS completion metadata stamp	
--	--	--	--	--	--

Stockout Notebook

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass/Fail
WB -M4 -01	Below-reorder flag — TRUE branch.	Insert: material_id='S01', quantity_on_hand=10, reorder_level=50.	is_below_reorder = true.	The result is the same as expected result	Pass
WB -M4 -02	Below-reorder flag — FALSE branch.	Insert: material_id='S02', quantity_on_hand=100, reorder_level=50.	is_below_reorder = false.	The result is the same as expected result	Pass
WB -M4 -03	Stockout risk — TRUE branch (cover < lead time).	Insert: stock_cover_days=5,	is_stockout_risk = true.	The result is the same as expected result	Pass

		lead_time_days=14.			
WB -M4 -04	Stockout risk — FALSE branch.	Insert: stock_cover_days=30, lead_time_days=14.	is_stockout_risk = false.	The result is the same as expected result	Pass
WB -M4 -05	Safe default when avg_daily_consumption is zero.	Insert material with no production history.	stock_cover_days = 9999.0 (no division-by-zero error).	The result is the same as expected result	Pass
WB -M4 -06	Q7 finished_product join normalises whitespace and case (Fix #5).	Insert sales row with finished_product=' PaintCoat ' and production row with 'PAINTCOAT'.	Join matches and row appears in impacted set (not dropped silently).	The result is the same as expected result	Pass
WB -M4 -07	Unmatched finished_products are surfaced, not silent.	Insert sales row with finished_product='UnknownProduct' that has no match.	Cell 4 prints this row in the 'Unmatched finished_product values' output.	The result is the same as expected result	Pass

WB -M4 -08	Merge material present in both M3 and M4 outputs.	Run Cell 6 after M3 publishes ra_mc_flags. Inspect combined for a material with both RA and stockout rows.	Single row in combined with both RA/MC flags AND stockout columns populated.	The result is the same as expected result	Pass
WB -M4 -09	Merge material in M3 only (M4 stockout missing).	Inject case: material present in ra_mc_flags but missing from stockout (e.g. no inventory snapshot).	Row still exists in combined; is_below_reorder/is_stockout_risk default to false (via fillna).	The result is the same as expected result	Pass
WB -M4 -10	Merge — material in M4 only (M3 RA/MC missing).	Inject case: material in stockout but missing from ra_mc_flags.	Row exists in combined; is_recoverable_asset / MC flags default to false.	The result is the same as expected result	Pass
WB -M4 -11	Combined output has all 16 expected columns.	Inspect combined.print Schema() output.	Schema contains material_id, snapshot_date, qty, RA/MC fields (5), stockout fields (4),	The result is the same as expected result	Pass

			and supporting columns.		
WB -M4 -12	Outputs written to all three silver paths.	Check portal for silver/stockout_risk/, silver/impacted_sales_orders/, silver/combined_inventory/.	All three folders exist and contain .parquet files.	The result is the same as expected result	Pass

Gold Views Logic

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass/Fail
WB-M6-01	vw_q1 filter logic — only below-reorder rows return.	SELECT COUNT(*) FROM vw_q1_below_reorder WHERE quantity_on_h and >= reorder_level;	Returns 0 (no row violates the filter).	Returned 0. No records breached the safety threshold parameters.	Pass
WB-M6-02	vw_q2 filter logic — verify MC classifications and defensive caps.	SELECT COUNT(*) FROM vw_q2_mc_provisions WHERE mc_provision_amount > inventory_value;	Returns 0 (proving the defensive safety cap prevents inflated losses).	Returned 0. All financial losses are successfully clamped inside asset boundaries.	Pass
WB-M6-03	vw_q3 filter logic — only MC Dead Stock rows return.	SELECT * FROM vw_q3_mc_dead_stock; — manually verify	Every returned row has months_since_consumed >= 9 and	Confirmed all rows accurately resolve to expired batch lot partitions.	Pass

		months_since_consumed >= 9 on every row.	mc_provision_amount > 0.		
WB-M6-04	vw_q4 filter logic — only MC Expired rows return.	SELECT * FROM vw_q4_mc_expired; verify each row has a passed expiry implicitly.	All returned rows are flagged is_mc_expired = 1 in underlying fact.	Confirmed all rows accurately resolve to expired batch lot partitions.	Pass
WB-M6-05	vw_q5 total matches backend notebook logs (cross-layer consistency).	SELECT SUM(mc_provision_amount) FROM vw_q5_provision;	Total provision sum matches the validated summary output from the Gold layer backend pipeline.	The calculated view sum reconciles perfectly with the backend pipeline logs after defensive caps are evaluated.	
WB-M6-06	vw_q7 join integrity — every material_id resolves to a material name.	SELECT COUNT(*) FROM vw_q7_impacted_orders WHERE material_name IS NULL;	Returns 0 (no orphaned data or broken lake joins).	Returned 0. All customer sales rows successfully mapped to master material names.	Pass

WB -M6 -07	Schema validation — all 6 views compile and return correct attributes.	Run SELECT TOP 5 * FROM each of the 6 views.	Each view returns structural data rows with the correct columns and zero SQL syntax faults.	All 6 views compiled successfully and initialized expected data schemas with zero faults.	Pass
------------------	--	--	---	---	------

5.5.4 User Acceptance Testing

User Acceptance Testing (UAT) is the project's final validation process that provides confirmation on the Power BI dashboard if it meets the business criteria specified for PPG's inventory risk management system. This test focuses on correctness to operational usability, ensuring that the dashboard visuals accurately represent the outputs from the gold layer Synapse SQL views.

Test ID	Test Description	Input / Steps	Expected Result	Actual Result	Pass /Fail
UAT-01	Page 1 correctly displays Total Financial Exposure KPI card.	Open Power BI dashboard. Navigate to Page 1 – Inventory Risk Overview. Apply no filters.	KPI card labelled Total Financial Exposure shows the sum of mc_provision_amount in RM millions format.	The KPI card displayed RM4.2M, correctly. The value is formatted in millions with the label visible below.	Pass

UAT-02	Page 1 Critical Storage Count card shows correct count of materials below reorder level.	Open Page 1. Observe the Critical Storage Count KPI card. Apply no filters.	Card shows the count of distinct material_id values where quantity_on_hand is below reorder_level.	Card returned a count of 39 materials with no filters applied.	Pass
UAT-03	Page 1 bar chart correctly compares current stock against reorder thresholds by category.	Open Page 1. Observe the Current Stock vs Reorder Thresholds clustered bar chart.	The chart displays the current stock and its reorder threshold.	Chart rendered correctly with four categories. All categories are below the reorder threshold.	Pass
UAT-04	Page 1 donut chart correctly segments quantity on hand by Risk Status.	Open Page 1. Observe the Sum of Quantity on Hand by Risk Status donut chart.	Donut chart shows three segments: Recoverable Asset (orange), Stockout Risk (red), and Healthy (green) with correct percentage labels.	Donut chart displayed Recoverable Asset at 74.21% (12K units), Stockout Risk at 20.46% (3K units), and Healthy at 5.33% (1K units), matching	Pass

				fact_inventory_status Risk Status field.	
UAT-05	Page 1 material detail table applies red conditional formatting to Stockout Risk rows.	Open Page 1. Scroll through the material detail table. Observe the Risk Status column formatting.	Could easily view different risk status.	All Stockout Risk rows displayed red background fills on the Risk Status column. Healthy with green background and Recoverable Asset rows showed orange background.	Pass
UAT-06	Verify that the detailed table shows the total cost of all PPG's assets while the main KPI card isolates the actual financial loss value.	Open Page 2 – MC Obsolescence Analysis. Look at the big KPI card value and compare it directly to the total number at the bottom of the table.	The table total should be higher than the KPI card because the table calculates the total cost of all materials, while the card only calculates specific financial losses.	The big KPI card correctly focuses only on the final risk loss of RM3,516,990.87, while the table bottom total accurately adds up the full material asset cost of RM4,186,539.75.	Pass

UAT-07	Page 2 donut chart correctly splits MC provision between Dead Stock and Expired categories.	Open Page 2. Observe the Financial Loss Split donut chart.	The donut chart shows two segments: MC Dead Stock in dark red and MC Expired in orange, with percentage labels visible.	Donut chart displayed MC Dead Stock at 96.08% and MC Expired at 3.92%. Colours matched the expected risk severity convention.	Pass
UAT-08	Page 2 High-Risk Obsolescence table lists materials sorted by inventory value descending.	Open Page 2. Observe the High-Risk Obsolescence Asset Details table. Check sort order.	The table should list MC-classified materials sorted from highest to lowest inventory_value, with material_id, material_name, category, inventory value, and obsolescence_type columns visible.	Table sorted correctly with Premium Violet Pigment (MAT053) at RM342,491.97 at the top, followed by Pure Yellow Paste (MAT052) at RM308,176.00.	Pass

UAT-09	Page 3 bar chart correctly ranks jeopardized customer orders by material category.	Open Page 3 – Supply Chain Stockout Alert. Observe the At-Risk Customer Orders by Material Category bar chart.	Horizontal bar chart shows categories sorted descending by count of sales_order_id. Pigment should appear at the top as the most affected category.	Bar chart rendered with Pigment at 1,082 orders, Packaging at 825, Solvent at 825, and Resin at 807. Chart was sorted in descending order and all bars were displayed in red-orange to reflect risk severity.	Pass
UAT-10	Page 3 At-Risk Sales Orders table displays correct stock cover days and lead time values for at-risk orders.	Open Page 3. Observe the At-Risk Customer Sales Orders Detail table. Check values for stock_cover_days and lead_time_days columns.	The table should list sales orders details with sales_order_id, customer_name, finished_product, required_delivery_date, stock_cover_days and lead_time_days. Orders should show stock cover days significantly lower than lead time days.	The table showed multiple orders including SO-003 for IJM Corporation and SO-013 for SP Setia Hardware with stock_cover_days of 79 days against lead_time_days of 163, confirming a critical replenishment gap. Date columns displayed correctly from dim_date.	Pass

5.5.1 Testing Summary & Sign-off

The summary and sign-off documents any failures done by each member which details on the testing type, total tests done, number of passes and number of fails and are validated by members assigned to that task.

Member	Testing Type	Total Tests	Passed	Failed	Member Signature / Date
M1	Black Box	5 (BB-M1-01 to 05)	5	0	ChohJingYi 8/6/2026
M2	Black Box	7 (BB-M2-01 to 07)	7	0	Evelyn 8/6/2026
M3	White Box	11 (WB-M3-01 to 11)	11	0	Dayang 8/6/2026
M4	White Box	12 (WB-M4-01 to 12)	12	0	Iman 8/6/2026
M5	Black Box	7 (BB-M5-01 to 07)	7	0	Nuraisyah 8/6/2026
M6	White Box	8 (WB-M6-01 to 08)	7	0	GuiKahSin 8/6/2026
M7 + Team	UAT	10 (UAT-01 to 10)	10	0	YingJia 8/6/2026

5.6 Summary

The implementation of the PPG Recoverable Assets and Inventory Risk Management system on Microsoft Azure follows a structured Medallion architecture to ensure clean, reliable data processing. In the bronze layer, Azure Data Factory ingests six core operational datasets into ADLS Gen2, using Mapping Data Flows to automatically separate clean records into verified folders while isolating duplicate data in a quarantine folder. In the silver layer layer RA and Magna Carta business rules were applied to evaluate excess stock, dead stock, expired materials, and financial provisions. Furthermore, items below reorder, stockout risks and impacted customer sales orders. Both implementations helped in answering all of the business questions.

The presentation layer contains seven pre-calculated Synapse SQL views containing all business logic which is used by Power BI for visualization. The dashboard includes an Inventory Risk Summary page, a MC Obsolescence Assessment page and a Supply Chain Stockout Notification page highlighting customer orders where stock cover days are alarmingly lower than supplier lead times.

The testing stage utilizes three testing types which are black box, white box and user acceptance testing (UAT) which has a total of 60 test cases. Black box testing helped confirmed ingestion, quarantine routing and loading of the gold table. White box testing helped in conforming business-logic paths in the PySpark notebooks and SQL views. UAT verified that all three dashboard pages showed correct KPIs and aligned figures with the upstream pipeline. All testing methods showed no failures indicating a successful pipeline implementation.

6.0 REFERENCES

- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021, January). Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In Proceedings of CIDR (Vol. 8, No. 1, p. 28). Sn.
- Chopra, S., & Meindl, P. (2001). Strategy, planning, and operation. *Supply Chain Management*, 15(5), 71-85.
- Daruvuri, R., Ravikumar, R., Mannem, P., & Aeniga, S. R. (2024). Augmenting Business Intelligence How AI and Data Engineering Elevate Power BI Analytics. *International Journal of Innovative Research in Computer and Communication Engineering*, 12(12), 13012-13022. [10.15680/IJRCCE.2024.1212002](https://doi.org/10.15680/IJRCCE.2024.1212002)
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling*. John Wiley & Sons.
- Reis, J., & Housley, M. (2022). *Fundamentals of data engineering: Plan and build robust data systems*. O'Reilly Media.
- Sharma, P., & Panda, S. (2023). Cloud computing for supply chain management and warehouse automation: a case study of azure cloud. *Int. J. Smart Sens. Adhoc Network*, 19-29. <https://doi.org/10.47893/IJSSAN.2023.1227>
- Silver, E. A., Pyke, D. F., & Thomas, D. J. (2016). *Inventory and production management in supply chains*. CRC press.
- Vassiliadis, P. (2009). A survey of extract–transform–load technology. *International Journal of Data Warehousing and Mining (IJDWM)*, 5(3), 1-27.
- PPG Industries. (n.d.). *Governance*. Retrieved May 2, 2026, from <https://www.ppg.com/en-US/about-ppg/governance>